

PROTOCOLS – DEVICE PATH PROTOCOL

This section contains the definition of the device path protocol and the information needed to construct and manage device paths in the UEFI environment. A device path is constructed and used by the firmware to convey the location of important devices, such as the boot device and console, consistent with the software-visible topology of the system.

10.1 Device Path Overview

A *Device Path* is used to define the programmatic path to a device. The primary purpose of a Device Path is to allow an application, such as an OS loader, to determine the physical device that the interfaces are abstracting.

A collection of device paths is usually referred to as a name space. ACPI, for example, is rooted around a name space that is written in ASL (ACPI Source Language). Given that EFI does not replace ACPI and defers to ACPI whenever possible, it would seem logical to utilize the ACPI name space in EFI. However, the ACPI name space was designed for usage at operating system runtime and does not fit well in platform firmware or OS loaders. Given this, EFI defines its own name space, called a *Device Path*.

A Device Path is designed to make maximum leverage of the ACPI name space. One of the key structures in the Device Path defines the linkage back to the ACPI name space. The Device Path also is used to fill in the gaps where ACPI defers to buses with standard enumeration algorithms. The Device Path is able to relate information about which device is being used on buses with standard enumeration mechanisms. The Device Path is also used to define the location on a medium where a file should be, or where it was loaded from. A special case of the Device Path can also be used to support the optional booting of legacy operating systems from legacy media.

The Device Path was designed so that the OS loader and the operating system could tell which devices the platform firmware was using as boot devices. This allows the operating system to maintain a view of the system that is consistent with the platform firmware. An example of this is a “headless” system that is using a network connection as the boot device and console. In such a case, the firmware will convey to the operating system the network adapter and network protocol information being used as the console and boot device in the device path for these devices.

10.2 EFI Device Path Protocol

This section provides a detailed description of `EFI_DEVICE_PATH_PROTOCOL`.

Summary

Can be used on any device handle to obtain generic path/location information concerning the physical device or logical device. If the handle does not logically map to a physical device, the handle may not necessarily support the device path protocol. The device path describes the location of the device the handle is for. The size of the Device Path can be determined from the structures that make up the Device Path.

GUID

```
#define EFI_DEVICE_PATH_PROTOCOL_GUID \
{0x09576e91, 0x6d3f, 0x11d2, \
 {0x8e, 0x39, 0x00, 0xa0, 0xc9, 0x69, 0x72, 0x3b}}
```

Protocol Interface Structure

```
/**
 * EFI_DEVICE_PATH_PROTOCOL
 */
typedef struct _EFI_DEVICE_PATH_PROTOCOL {
    UINT8      Type;
    UINT8      SubType;
    UINT8      Length[2];
} EFI_DEVICE_PATH_PROTOCOL;
```

Description

The executing UEFI Image may use the device path to match its own device drivers to the particular device. Note that the executing UEFI OS loader and UEFI application images must access all physical devices via Boot Services device handles until [EFI_BOOT_SERVICES.ExitBootServices\(\)](#) is successfully called. A UEFI driver may access only a physical device for which it provides functionality.

10.3 Device Path Nodes

There are six major types of Device Path nodes:

- **Hardware Device Path.** This Device Path defines how a device is attached to the resource domain of a system, where resource domain is simply the shared memory, memory mapped I/O, and I/O space of the system.
- **ACPI Device Path.** This Device Path is used to describe devices whose enumeration is not described in an industry-standard fashion. These devices must be described using ACPI AML in the ACPI name space; this Device Path is a linkage to the ACPI name space.
- **Messaging Device Path.** This Device Path is used to describe the connection of devices outside the resource domain of the system. This Device Path can describe physical messaging information such as a SCSI ID, or abstract information such as networking protocol IP addresses.
- **Media Device Path.** This Device Path is used to describe the portion of a medium that is being abstracted by a boot service. For example, a Media Device Path could define which partition on a hard drive was being used.
- **BIOS Boot Specification Device Path.** This Device Path is used to point to boot legacy operating systems; it is based on the BIOS Boot Specification Version 1.01. See [References](#) for details on obtaining this specification.
- **End of Hardware Device Path.** Depending on the Sub-Type, this Device Path node is used to indicate the end of the Device Path instance or Device Path structure.

10.3.1 Generic Device Path Structures

A Device Path is a variable-length binary structure that is made up of variable-length generic Device Path nodes. *Generic Device Path Node Structure* defines the structure of a variable-length generic Device Path node and the lengths of its components. The table (below) defines the type and sub-type values corresponding to the Device Paths described in *Device Path Nodes*; all other type and sub-type values are Reserved.

Table 10.1: Generic Device Path Node Structure

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 0x01 - Hardware Device Path Type 0x02 - ACPI Device Path Type 0x03 - Messaging Device Path Type 0x04 - Media Device Path Type 0x05 - BIOS Boot Specification Device Path Type 0x7F - End of Hardware Device Path
Sub-Type	1	1	Sub-Type - Varies by Type. (See table below)
Length	2	2	Length of this structure in bytes. Length is 4 + <i>n</i> bytes.
Specific Device Path Data	4	<i>n</i>	Specific Device Path data. Type and Sub-Type define type of data. Size of data is included in Length.

A Device Path is a series of generic Device Path nodes. The first Device Path node starts at byte offset zero of the Device Path. The next Device Path node starts at the end of the previous Device Path node. Therefore all nodes are byte-packed data structures that may appear on any byte boundary. All code references to device path nodes must assume all fields are unaligned. Since every Device Path node contains a length field in a known place, it is possible to traverse Device Path nodes that are of an unknown type. There is no limit to the number, type, or sequence of nodes in a Device Path.

A Device Path is terminated by an End of Hardware Device Path node. This type of node has two sub-types (*Device Path End Structure*):

- *End This Instance of a Device Path* (sub-type 0x01). This type of node terminates one Device Path instance and denotes the start of another. This is only required when an environment variable represents multiple devices. An example of this would be the *ConsoleOut* environment variable that consists of both a VGA console and serial output console. This variable would describe a console output stream that is sent to both VGA and serial concurrently and thus has a Device Path that contains two complete Device Paths.
- *End Entire Device Path* (sub-type 0xFF). This type of node terminates an entire Device Path. Software searches for this sub-type to find the end of a Device Path. All Device Paths must end with this sub-type.

Table 10.2: Device Path End Structure

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 0x7F - End of Hardware Device Path
Sub-Type	1	1	Sub-Type 0xFF - End Entire Device Path, or Sub-Type 0x01 - End This Instance of a Device Path and start a new Device Path
Length	2	2	Length of this structure in bytes. Length is 4 bytes.

10.3.2 Hardware Device Path

This Device Path defines how a device is attached to the resource domain of a system, where resource domain is simply the shared memory, memory mapped I/O, and I/O space of the system. It is possible to have multiple levels of Hardware Device Path such as a PCCARD device that was attached to a PCCARD PCI controller.

10.3.2.1 PCI Device Path

The Device Path for PCI defines the path to the PCI configuration space address for a PCI device. There is one PCI Device Path entry for each device and function number that defines the path from the root PCI bus to the device. Because the PCI bus number of a device may potentially change, a flat encoding of single PCI Device Path entry cannot be used. An example of this is when a PCI device is behind a bridge, and one of the following events occurs:

- OS performs a Plug and Play configuration of the PCI bus.
- A hot plug of a PCI device is performed.
- The system configuration changes between reboots.

The PCI Device Path entry must be preceded by an ACPI Device Path entry that uniquely identifies the PCI root bus. The programming of root PCI bridges is not defined by any PCI specification and this is why an ACPI Device Path entry is required.

Table 10.3: PCI Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 1 - Hardware Device Path
Sub-Type	1	1	Sub-Type 1 - PCI
Length	2	2	Length of this structure is 6 bytes
Function	4	1	PCI Function Number
Device	5	1	PCI Device Number

10.3.2.2 PCCARD Device Path

Pccard Device Path

Table 10.4: PCCARD Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 1 - Hardware Device Path
Sub-Type	1	1	Sub-Type 2 - PCCARD
Length	2	2	Length of this structure in bytes. Length is 5 bytes.
Function Number	4	1	Function Number (0 = First Function)

10.3.2.3 Memory Mapped Device Path

Table 10.5: Memory Mapped Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 1 - Hardware Device Path.
Sub-Type	1	1	Sub-Type 3 - Memory Mapped.
Length	2	2	Length of this structure in bytes. Length is 24 bytes.
Memory Type	4	4	EFI_MEMORY_TYPE. Type EFI_MEMORY_TYPE is defined in the <i>EFI_BOOT_SERVICES.AllocatePages()</i> function description.
Start Address	8	8	Starting Memory Address.
End Address	16	8	Ending Memory Address.

10.3.2.4 Vendor Device Path

The Vendor Device Path allows the creation of vendor-defined Device Paths. A vendor must allocate a Vendor GUID for a Device Path. The Vendor GUID can then be used to define the contents on the n bytes that follow in the Vendor Device Path node.

Table 10.6: Vendor-Defined Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 1 - Hardware Device Path.
Sub-Type	1	1	Sub-Type 4 - Vendor.
Length	2	2	Length of this structure in bytes. Length is 20 + n bytes.
Vendor_GUID	4	16	Vendor-assigned GUID that defines the data that follows.
Vendor Defined Data	20	n	Vendor-defined variable size data.

10.3.2.5 Controller Device Path

Table 10.7: Controller Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 1 - Hardware Device Path.
Sub-Type	1	1	Sub-Type 5 - Controller.
Length	2	2	Length of this structure in bytes. Length is 8 bytes.
Controller Number	4	4	Controller number.

10.3.2.6 BMC Device Path

The Device Path for a Baseboard Management Controller (BMC) host interface.

Table 10.8: BMC Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 1 - Hardware Device Path.
Sub-Type	1	1	Sub Type 6 - BMC
Length	2	2	Length of this structure in bytes. Length is 13 bytes.
Interface Type	4	1	The Baseboard Management Controller (BMC) host interface type: 0x00: Unknown 0x01: KCS: Keyboard Controller Style 0x02: SMIC: Server Management Interface Chip 0x03: BT: Block Transfer
Base Address	5	8	Base address (either memory-mapped or I/O) of the BMC. If the least-significant bit of the field is a 1, the address is in I/O space; otherwise, the address is memory-mapped. Refer to the IPMI Interface Specification for usage details.

10.3.3 ACPI Device Path

This Device Path contains ACPI Device IDs that represent a device's Plug and Play Hardware ID and its corresponding unique persistent ID. The ACPI IDs are stored in the ACPI _HID, _CID, and _UID device identification objects that are associated with a device. The ACPI Device Path contains values that must match exactly the ACPI name space that is provided by the platform firmware to the operating system. Refer to the ACPI specification for a complete description of the _HID, _CID, and _UID device identification objects.

The _HID and _CID values are optional device identification objects that appear in the ACPI name space. If only _HID is present, the _HID must be used to describe any device that will be enumerated by the ACPI driver. The _CID, if present, contains information that is important for the OS to attach generic driver (e.g., PCI Bus Driver), while the _HID contains information important for the OS to attach device-specific driver. The ACPI bus driver only enumerates a device when no standard bus enumerator exists for a device.

The _UID object provides the OS with a serial number-style ID for a device that does not change across reboots. The object is optional, but is required when a system contains two devices that report the same _HID. The _UID only needs to be unique among all device objects with the same _HID value. If no _UID exists in the ACPI name space for a _HID the value of zero must be stored in the _UID field of the ACPI Device Path.

The ACPI Device Path is only used to describe devices that are not defined by a Hardware Device Path. An _HID (along with _CID if present) is required to represent a PCI root bridge, since the PCI specification does not define the programming model for a PCI root bridge. There are two subtypes of the ACPI Device Path: a simple subtype that only includes the _HID and _UID fields, and an extended subtype that includes the _HID, _CID, and _UID fields.

The ACPI Device Path node only supports numeric 32-bit values for the _HID and _UID values. The Expanded ACPI Device Path node supports both numeric and string values for the _HID, _UID, and _CID values. As a result, the ACPI Device Path node is smaller and should be used if possible to reduce the size of device paths that may potentially be stored in nonvolatile storage. If a string value is required for the _HID field, or a string value is required for the _UID field, or a _CID field is required, then the Expanded ACPI Device Path node must be used. If a string field of the Expanded ACPI Device Path node is present, then the corresponding numeric field is ignored.

The _HID and _CID fields in the ACPI Device Path node and Expanded ACPI Device Path node are stored as a 32-bit compressed EISA-type IDs. The following macro can be used to compute these EISA-type IDs from a Plug and Play Hardware ID. The Plug and Play Hardware IDs used to compute the _HID and _CID fields in the EFI device path nodes must match the Plug and Play Hardware IDs used to build the matching entries in the ACPI tables. The compressed EISA-type IDs produced by this macro differ from the compressed EISA-type IDs stored in ACPI tables. As a result,

the compressed EISA-type IDs from the ACPI Device Path nodes cannot be directly compared to the compressed EISA-type IDs from the ACPI table.

```
#define EFI_PNP_ID(ID) (UINT32)((ID) << 16) | 0x41D0)
#define EISA_PNP_ID(ID) EFI_PNP_ID(ID)
```

Table 10.9: ACPI Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 2 - ACPI Device Path.
Sub-Type	1	1	Sub-Type 1 ACPI Device Path.
Length	2	2	Length of this structure in bytes. Length is 12 bytes.
_HID	4	4	Device's PnP hardware ID stored in a numeric 32-bit compressed EISA-type ID. This value must match the corresponding _HID in the ACPI name space.
_UID	8	4	Unique ID that is required by ACPI if two devices have the same _HID. This value must also match the corresponding _UID/_HID pair in the ACPI name space. Only the 32-bit numeric value type of _UID is supported; thus, strings must not be used for the _UID in the ACPI name space.

Table 10.10: Expanded ACPI Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 2 - ACPI Device Path.
Sub-Type	1	1	Sub-Type 2 Expanded ACPI Device Path.
Length	2	2	Length of this structure in bytes. Minimum length is 19 bytes. The actual size will depend on the size of the _HIDSTR, _UIDSTR, and _CIDSTR fields.
_HID	4	4	Device's PnP hardware ID stored in a numeric 32-bit compressed EISA-type ID. This value must match the corresponding _HID in the ACPI name space.
_UID	8	4	Unique ID that is required by ACPI if two devices have the same _HID. This value must also match the corresponding _UID/_HID pair in the ACPI name space.
_CID	12	4	Device's compatible PnP hardware ID stored in a numeric 32-bit compressed EISA-type ID. This value must match at least one of the compatible device IDs returned by the corresponding _CID in the ACPI name space.
_HIDSTR	16	>=1	Device's PnP hardware ID stored as a null-terminated ASCII string. This value must match the corresponding _HID in the ACPI name space. If the length of this string not including the null-terminator is 0, then the _HID field is used. If the length of this null-terminated string is greater than 0, then this field supersedes the _HID field.

continues on next page

Table 10.10 – continued from previous page

_UIDSTR	Varies	≥ 1	Unique ID that is required by ACPI if two devices have the same _HID. This value must also match the corresponding _UID/_HID pair in the ACPI name space. This value is stored as a null-terminated ASCII string. If the length of this string not including the null-terminator is 0, then the _UID field is used. If the length of this null-terminated string is greater than 0, then this field supersedes the _UID field. The Byte Offset of this field can be computed by adding 16 to the size of the _HIDSTR field.
_CIDSTR	Varies	≥ 1	Device's compatible PnP hardware ID stored as a null-terminated ASCII string. This value must match at least one of the compatible device IDs returned by the corresponding _CID in the ACPI name space. If the length of this string not including the null-terminator is 0, then the _CID field is used. If the length of this null-terminated string is greater than 0, then this field supersedes the _CID field. The Byte Offset of this field can be computed by adding 16 to the sum of the sizes of the _HIDSTR and _UIDSTR fields.

10.3.3.1 ACPI _ADR Device Path

The _ADR device path is used to contain video output device attributes to support the Graphics Output Protocol. The device path can contain multiple _ADR entries if multiple video output devices are displaying the same output.

Table 10.11: ACPI _ADR Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 2 - ACPI Device Path
Sub-Type	1	1	Sub-Type3 _ADR Device Path
Length	2	2	Length of this structure in bytes. Minimum length is 8.
_ADR	4	4	_ADR value. For video output devices the value of this field comes from Table B-2 ACPI 3.0 specification. At least one _ADR value is required
Additional _ADR	8	N	This device path may optionally contain more than one _ADR entry.

10.3.3.2 NVDIMM Device Path

This device path describes an NVDIMM device using the ACPI 6.0 specification defined NFIT Device Handle as the identifier.

Table 10.12: NVDIMM Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 2 - ACPI Device Path
Sub-Type	1	1	Sub-type 4 - NVDIMM Device
Length	2	2	8 - Single NFIT Device Handle is supported.
NFIT Device Handle	4	4	NFIT Device Handle - Unique physical identifier. See ACPI Defined Devices and Device Specific Objects section, NVDIMM Devices sub-chapter for the specific definition of the fields utilized for this handle.

continues on next page

Table 10.12 – continued from previous page

10.3.4 Messaging Device Path

This Device Path is used to describe the connection of devices outside the resource domain of the system. This Device Path can describe physical messaging information like SCSI ID, or abstract information like networking protocol IP addresses.

10.3.4.1 ATAPI Device Path

Table 10.13: ATAPI Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 1 - ATAPI
Length	2	2	Length of this structure in bytes. Length is 8 bytes.
PrimarySecondary	4	1	Set to zero for primary or one for secondary
SlaveMaster	5	1	Set to zero for master or one for slave mode
Logical Unit Number	6	2	Logical Unit Number

10.3.4.2 SCSI Device Path

Table 10.14: SCSI Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 2 - SCSI
Length	2	2	Length of this structure in bytes. Length is 8 bytes.
Target ID	4	2	Target ID on the SCSI bus (PUN)
Logical Unit Number	6	2	Logical Unit Number (LUN)

10.3.4.3 Fibre Channel Device Path

Table 10.15: Fibre Channel Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 3 - Fibre Channel
Length	2	2	Length of this structure in bytes. Length is 24 bytes.
Reserved	4	4	Reserved
World Wide Name	8	8	Fibre Channel World Wide Name

continues on next page

Table 10.15 – continued from previous page

Logical Unit Number	16	8	Fibre Channel Logical Unit Number
---------------------	----	---	-----------------------------------

Table 10.16: Fibre Channel Ex Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 21 - Fibre Channel Ex
Length	2	2	Length of this structure in bytes. Length is 24 bytes.
Reserved	4	4	Reserved
World Wide Name	8	8	8 byte array containing Fibre Channel End Device Port Name (a.k.a., World Wide Name)
Logical Unit Number	16	8	8 byte array containing Fibre Channel Logical Unit Number

The Fibre Channel Ex device path clarifies the definition of the Logical Unit Number field to conform with the T-10 SCSI Architecture Model 4 specification. The 8 byte Logical Unit Number field in the device path must conform with a logical unit number returned by a SCSI REPORT LUNS command.

When the Fibre Channel Ex Device Path is used with the Extended SCSI Pass Thru Protocol the UINT64 LUN argument must be converted to the eight byte array Logical Unit Number field in the device path by treating the eight byte array as an EFI UINT64. For example a Logical Unit Number array of { 0,1,2,3,4,5,6,7 } becomes a UINT64 of 0x0706050403020100.

When an application client displays or otherwise makes a 64-bit LUN visible to a user, it should be done in conformance with SAM-4. SAM-4 requires a LUN to be displayed in hexadecimal format with byte 0 first (i.e., on the left) and byte 7 last (i.e., on the right) regardless of the internal representation of the LUN. UEFI defines all data structures a “little endian” and SCSI defines all data structures as “big endian”. Fibre Channel Ex Device Path Example shows an example device path for a Fibre Channel controller on a typical UEFI platform. This Fibre Channel Controller is connected to the port 0 of the root hub, and its interface number is 0. The Fibre Channel Host Controller is a PCI device whose PCI device number 0x1F and PCI function 0x00. So, the whole device path for this Fibre Channel Controller consists an ACPI Device Path Node, a PCI Device Path Node, a Fibre Channel Device Path Node and a Device Path End Structure. The _HID and _UID must match the ACPI table description of the PCI Root Bridge. The Fibre Channel WWN and LUN were picked to show byte order and they are not typical real world values. The shorthand notation for this device path is:

PciRoot(0)/PCI(31,0)/FibreEx(0x0001020304050607, 0x0001020304050607)

Table 10.17: Fibre Channel Ex Device Path Example

Byte Offset	Byte Length	Data	Description
0	1	0x02	Generic Device Path Header - Type ACPI Device Path
1	1	0x01	Sub type - ACPI Device Path
2	2	0x0C	Length - 0x0C bytes
4	4	0x41D0, 0x0A03	_HID PNP0A03 - 0x41D0 represents the compressed string ‘PNP’ and is encoded in the low order bytes. The compression method is described in the ACPI Specification.
8	4	0x0000	_UID
12	1	0x01	Generic Device Path Header - Type Hardware Device Path
13	1	0x01	Sub type - PCI

continues on next page

Table 10.17 – continued from previous page

14	2	0x06	Length - 0x06 bytes
16	1	0x0	PCI Function
17	1	0x1F	PCI Device
18	1	0x03	Generic Device Path Header - Type Message Device Path
19	1	0x15	Sub type - Fibre Channel Ex
20	2	0x14	Length - 20 bytes
21	1	0x00	8 byte array containing Fibre Channel End Device Port Name (a.k.a., World Wide Name)
22	1	0x01	
23	1	0x02	
24	1	0x03	
25	1	0x04	
26	1	0x05	
27	1	0x06	
28	1	0x07	
29	1	0x00	8 byte array containing Fibre Channel Logical Unit Number
30	1	0x01	
31	1	0x02	
32	1	0x03	
33	1	0x04	
34	1	0x05	
35	1	0x06	
36	1	0x07	
37	1	0x7F	Generic Device Path Header - Type End of Hardware Device Path
38	1	0xFF	Sub type - End of Entire Device Path
39	2	0x04	Length - 0x04 bytes

10.3.4.4 1394 Device Path

Table 10.18: 1394 Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 4 - 1394
Length	2	2	Length of this structure in bytes. Length is 16 bytes.
Reserved	4	4	Reserved
GUID ¹	8	8	1394 Global Unique ID (GUID) ¹

Note: ¹ The usage of the term GUID is per the 1394 specification. This is not the same as the EFI_GUID type defined in the EFI Specification.

10.3.4.5 USB Device Paths

Table 10.19: USB Device Paths

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 5 - USB
Length	2	2	Length of this structure in bytes. Length is 6 bytes.
USB Parent Port Number	4	1	USB Parent Port Number
Interface	5	1	USB Interface Number

10.3.4.5.1 USB Device Path Example

USB Device Path Example shows an example device path for a USB controller on a desktop platform. This USB Controller is connected to the port 0 of the root hub, and its interface number is 0. The USB Host Controller is a PCI device whose PCI device number 0x1F and PCI function 0x02. So, the whole device path for this USB Controller consists an ACPI Device Path Node, a PCI Device Path Node, a USB Device Path Node and a Device Path End Structure. The `_HID` and `_UID` must match the ACPI table description of the PCI Root Bridge. The shorthand notation for this device path is:

PciRoot(0)/PCI(31,2)/USB(0,0).

Table 10.20: USB Device Path Examples

Byte Off-set	Byte length	Data	Description
0x00	0x01	0x02	Generic Device Path Header - Type ACPI Device Path
0x01	0x01	0x01	Sub type - ACPI Device Path
0x02	0x02	0x0C	Length - 0x0C bytes
0x04	0x04	0x41D0, 0x0A03	<code>_HID</code> PNP0A03 - 0x41D0 represents the compressed string 'PNP' and is encoded in the low order bytes. The compression method is described in the ACPI Specification.
0x08	0x04	0x0000	<code>_UID</code>
0x0C	0x01	0x01	Generic Device Path Header - Type Hardware Device Path
0x0D	0x01	0x01	Sub type - PCI
0x0E	0x02	0x06	Length - 0x06 bytes
0x10	0x01	0x02	PCI Function
0x11	0x01	0x1F	PCI Device
0x12	0x01	0x03	Generic Device Path Header - Type Message Device Path
0x13	0x01	0x05	Sub type - USB
0x14	0x02	0x06	Length - 0x06 bytes
0x16	0x01	0x00	Parent Hub Port Number
0x17	0x01	0x00	Controller Interface Number
0x18	0x01	0x7F	Generic Device Path Header - Type End of Hardware Device Path
0x19	0x01	0xFF	Sub type - End of Entire Device Path
0x1A	0x02	0x04	Length - 0x04 bytes

Another example is a USB Controller (interface number 0) that is connected to port 3 of a USB Hub Controller (interface number 0), and this USB Hub Controller is connected to the port 1 of the root hub. The shorthand notation for this device path is:

PciRoot(0)/PCI(31,2)/USB(1,0)/USB(3,0).

See table (below) showing the device path for this USB Controller.

Table 10.21: Another USB Device Path Example

Byte Off-set	Byte length	Data	Description
0x00	0x01	0x02	Generic Device Path Header - Type ACPI Device Path
0x01	0x01	0x01	Sub type - ACPI Device Path
0x02	0x02	0x0C	Length - 0x0C bytes
0x04	0x04	0x41D0, 0x0A03	_HID PNP0A03 - 0x41D0 represents the compressed string 'PNP' and is encoded in the low order bytes. The compression method is described in the ACPI Specification.
0x08	0x04	0x0000	_UID
0x0C	0x01	0x01	Generic Device Path Header - Type Hardware Device Path
0x0D	0x01	0x01	Sub type - PCI
0x0E	0x02	0x06	Length - 0x06 bytes
0x10	0x01	0x02	PCI Function
0x11	0x01	0x1F	PCI Device
0x12	0x01	0x03	Generic Device Path Header - Type Message Device Path
0x13	0x01	0x05	Sub type - USB
0x14	0x02	0x06	Length - 0x06 bytes
0x16	0x01	0x01	Parent Hub Port Number
0x17	0x01	0x00	Controller Interface Number
0x18	0x01	0x03	Generic Device Path Header - Type Message Device Path
0x19	0x01	0x05	Sub type - USB
0x1A	0x02	0x06	Length - 0x06 bytes
0x1C	0x01	0x03	Parent Hub Port Number
0x1D	0x01	0x00	Controller Interface Number
0x1E	0x01	0x7F	Generic Device Path Header - Type End of Hardware Device Path
0x1F	0x01	0xFF	Sub type - End of Entire Device Path
0x20	0x02	0x04	Length - 0x04 bytes

10.3.4.6 SATA Device Path

Table 10.22: SATA Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 18 - SATA
Length	2	2	Length of this structure in bytes. Length is 10 bytes.
HBA Port Number	4	2	The HBA port number that facilitates the connection to the device or a port multiplier. The value 0xFFFF is reserved.
Port Multiplier Port Number	6	2	The Port multiplier port number that facilitates the connection to the device. Must be set to 0xFFFF if the device is directly connected to the HBA.
Logical Unit Number	8	2	Logical Unit Number.

10.3.4.7 USB Device Paths (WWID)

This device path describes a USB device using its serial number.

Specifications, such as the USB Mass Storage class, bulk-only transport subclass, require that some portion of the suffix of the device's serial number be unique with respect to the vendor and product id for the device. So, in order to avoid confusion and overlap of WWID's, the interface's class, subclass, and protocol are included.

Table 10.23: USB WWID Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 16- USB WWID
Length	2	2	Length of this structure in bytes. Length is 10+
• Interface Number	4	2	USB interface number
• Device Vendor Id	6	2	USB vendor id of the device
• Device Product Id	8	2	USB product id of the device
• Serial Number	10	n	Last 64-or-fewer UTF-16 characters of the USB serial number. The length of the string is determined by the Length field less the offset of the Serial Number field (10)

Devices that do not have a serial number string must use the USB Device Path (type 5) as described in *USB Device Path Example*.

Including the interface as part of this node allows distinction for multi-interface devices, e.g., an HID interface and a Mass Storage interface on the same device, or two Mass Storage interfaces.

Load Option Processing defines special rules for processing the USB WWID Device Path. These special rules enable a device location to change and still have the system boot from the device.

10.3.4.8 Device Logical Unit

For some classes of devices, such as USB Mass Storage, it is necessary to specify the Logical Unit Number (LUN), since a single device may have multiple logical units. In order to boot from one of these logical units of the device, the Device Logical Unit device node is appended to the device path. The EFI path node subtype is defined, as in the Table below.

Table 10.24: Device Logical Unit

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 17 - Device Logical unit
Length	2	2	Length of this structure in bytes. Length is 5
LUN	4	1	Logical Unit Number for the interface

Load Option Processing defines special rules for processing the USB Class Device Path. These special rules enable a device location to change and still have the system recognize the device.

Globally Defined Variables defines how the *ConIn*, *ConOut*, and *ErrOut* variables are processed and contains special rules for processing the USB Class device path. These special rules allow all USB keyboards to be specified as valid input devices.

Table 10.25: USB Class Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path.
Sub-Type	1	1	Sub-Type 15 - USB Class.
Length	2	2	Length of this structure in bytes. Length is 11 bytes.
Vendor ID	4	2	Vendor ID assigned by USB-IF. A value of 0xFFFF will match any Vendor ID.
Product ID	6	2	Product ID assigned by USB-IF. A value of 0xFFFF will match any Product ID.
Device Class	8	1	The class code assigned by the USB-IF. A value of 0xFF will match any class code.
Device Subclass	9	1	The subclass code assigned by the USB-IF. A value of 0xFF will match any subclass code.
Device Protocol	10	1	The protocol code assigned by the USB-IF. A value of 0xFF will match any protocol code.

10.3.4.9 I₂O Device Path

Table 10.26: I₂O Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 6 - I2O Random Block Storage Class
Length	2	2	Length of this structure in bytes. Length is 8 bytes.
TID	4	4	Target ID (TID) for a device

10.3.4.10 MAC Address Device Path

Table 10.27: MAC Address Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 11 - MAC Address for a network interface
Length	2	2	Length of this structure in bytes. Length is 37 bytes.
MAC Address	4	32	The MAC address for a network interface padded with 0s
IfType	36	1	Network interface type(i.e., 802.3, FDDI). See RFC 3232

10.3.4.11 IPv4 Device Path

Previous versions of the specification only defined a 19 byte IPv4 device path. To access fields at off-set 19 or greater, the size of the device path must be checked first.

Table 10.28: IPv4 Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 12 - IPv4
Length	2	2	Length of this structure in bytes. Length is 27 bytes.
Local IP Address	4	4	The local IPv4 address
Remote IP Address	8	4	The remote IPv4 address
Local Port	12	2	The local port number
Remote Port	14	2	The remote port number
Protocol	16	2	The network protocol(i.e., UDP, TCP). See RFC 3232
StaticIPAddress	18	1	0x00 - The Source IP Address was assigned though DHCP 0x01 - The Source IP Address is statically bound
GatewayIPAddress	19	4	The Gateway IP Address
Subnet Mask	23	4	Subnet mask

10.3.4.12 IPv6 Device Path

Table 10.29: IPv6 Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 13 - IPv6
Length	2	2	Length of this structure in bytes. Length is 60 bytes.
Local IP Address	4	16	The local IPv6 address
Remote IP Address	20	16	The remote IPv6 address
Local Port	36	2	The local port number
Remote Port	38	2	The remote port number
Protocol	40	2	The network protocol (i.e., UDP, TCP). See RFC 3232
IPAddressOrigin	42	1	0x00 - The Local IP Address was manually configured. 0x01 - The Local IP Address is assigned through IPv6 stateless auto -configuration. 0x02 - The Local IP Address is assigned through IPv6stateful configuration.
PrefixLength	43	1	The Prefix Length
GatewayIPAddress	44	16	The Gateway IP Address

10.3.4.13 2. VLAN device path node

Table 10.30: VLAN device path node

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 20 - Vlan (802.1q)
Length	2	2	Length of this device node
Vlanid	4	2	VLAN identifier (0-4094)

10.3.4.14 InfiniBand Device Path

Table 10.31: InfiniBand Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 9 - InfiniBand
Length	2	2	Length of this structure in bytes. Length is 48 bytes.
Resource Flags	4	4	Flags to help identify/manage InfiniBand device path elements: • Bit 0: IOC/Service (0b = IOC, 1b = Service) • Bit 1: Extend Boot Environment • Bit 2: Console Protocol • Bit 3: Storage Protocol • Bit 4: Network Protocol All other bits are reserved.
PORT GUID	8	16	128-bit Global Identifier for remote fabric port
IOC GUID/Service ID	24	8	64-bit unique identifier to remote IOC or server process. Interpretation of field specified by Resource Flags (bit 0)
Target Port ID	32	8	64-bit persistent ID of remote IOC port
Device ID	40	8	64-bit persistent ID of remote device

Note: The usage of the terms GUID and GID is per the InfiniBand Specification. The term GUID is not the same as the EFI_GUID type defined in this EFI Specification.

10.3.4.15 UART Device Path

Table 10.32: UART Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 – Messaging Device Path
Sub-Type	1	1	Sub-Type 14 – UART
Length	2	2	Length of this structure in bytes. Length is 19 bytes.

continues on next page

Table 10.32 – continued from previous page

Reserved	4	4	Reserved
Baud Rate	8	8	The baud rate setting for the UART style device. A value of 0 means that the device's default baud rate will be used.
Data Bits	16	1	The number of data bits for the UART style device. A value of 0 means that the device's default number of data bits will be used.
Parity	17	1	The parity setting for the UART style device. Parity 0x00 - Default Parity Parity 0x01 - No Parity Parity 0x02 - Even Parity Parity 0x03 - Odd Parity Parity 0x04 - Mark Parity Parity 0x05 - Space Parity
Stop Bits	18	1	The number of stop bits for the UART style device. Stop Bits 0x00 - Default Stop Bits Stop Bits 0x01 - 1 Stop Bit Stop Bits 0x02 - 1.5 Stop Bits Stop Bits 0x03 - 2 Stop Bits

10.3.4.16 Vendor-Defined Messaging Device Path

Table 10.33: Vendor-Defined Messaging Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 10 - Vendor
Length	2	2	Length of this structure in bytes. Length is 20 + n bytes.
Vendor GUID	4	16	Vendor-assigned GUID that defines the data that follows
Vendor Defined Data	20	n	Vendor-defined variable size data

The following GUIDs are used with a Vendor-Defined Messaging Device Path to describe the transport protocol for use with PC-ANSI, VT-100, VT-100+, and VT-UTF8 terminals. Device paths can be constructed with this node as the last node in the device path. The rest of the device path describes the physical device that is being used to transmit and receive data. The PC-ANSI, VT-100, VT-100+, and VT-UTF8 GUIDs define the format of the data that is being sent through the physical device. Additional GUIDs can be generated to describe additional transport protocols.

```
#define EFI_PC_ANSI_GUID\
{0xe0c14753, 0xf9be, 0x11d2, {0x9a, 0x0c, 0x00, 0x90, 0x27, 0x3f, 0xc1, 0x4d}}

#define EFI_VT_100_GUID\
{0xdfa66065, 0xb419, 0x11d3, {0x9a, 0x2d, 0x00, 0x90, 0x27, 0x3f, 0xc1, 0x4d}}

#define EFI_VT_100_PLUS_GUID\
{0x7baec70b, 0x57e0, 0x4c76, {0x8e, 0x87, 0x2f, 0x9e, 0x28, 0x08, 0x83, 0x43}}
```

(continues on next page)

(continued from previous page)

```
#define EFI_VT_UTF8_GUID\
{0xad15a0d6,0x8bec,0x4acf,{0xa0,0x73,0xd0,0x1d,0xe7,0x7e,0x2d,0x88}}
```

10.3.4.17 UART Flow Control Messaging Path

The UART messaging device path defined in the EFI 1.02 specification does not contain a provision for flow control. Therefore, a new device path node is needed to declare flow control characteristics. It is a vendor-defined messaging node which may be appended to the UART node in a device path. It has the following definition:

```
#define DEVICE_PATH_MESSAGING_UART_FLOW_CONTROL \
{0x37499a9d,0x542f,0x4c89,{0xa0,0x26,0x35,0xda,0x14,0x20,0x94,0xe4}}
```

Table 10.34: UART Flow Control Messaging Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 10 - Vendor
Length	2	2	Length of this structure in bytes. Length is 24 bytes.
Vendor GUID	4	16	DEVICE_PATH_MESSAGING_UART_FLOW_CONTROL
Flow_Control_Map	20	4	Bitmap of supported flow control types. • Bit 0 set indicates hardware flow control. • Bit 1 set indicates Xon/Xoff flow control. • All other bits are reserved and are clear.

A debugport driver that implements Xon/Xoff flow control would produce a device path similar to the following:

```
PciRoot(0)/Pci(0x1f,0)/ACPI(PNP0501,0)/UART(115200,N,8,1)/UartFlowCtrl(2)/DebugPort()
```

Note: If no bits are set in the Flow_Control_Map, this indicates there is no flow control and is equivalent to leaving the flow control node out of the device path completely.

10.3.4.18 Serial Attached SCSI (SAS) Device Path

This section defines the device node for Serial Attached SCSI (SAS) devices.

Table 10.35: SAS Messaging Device Path Structure

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type -3 Messaging
Sub Type	1	1	10 (Vendor)
Length	2	2	Length of this Structure.
Vendor GUID	4	16	d487dd b4-008b-11d9-af dc-001083ffa4d
Reserved	20	4	Reserved for future use.

continues on next page

Table 10.35 – continued from previous page

SAS Address	24	8	SAS Address for Serial Attached SCSI Target.
Logical Unit Number	32	8	SAS Logical Unit Number.
SAS/SATA device and Topology Info	40	2	More Information about the device and its interconnect
Relative Target Port	42	2	Relative Target Port (RTP)

Summary

The device node represented by the structure in the table above shall be appended after the Hardware Device Path node in the device path.

There are two cases for boot devices connected with SAS HBA's. Each of the cases is described below with an example of the expected Device Path for these.

- SAS Device anywhere in an SAS domain accessed through SSP Protocol.

```
PciRoot(0)/PCI(1,0)/Sas(0x31000004CF13F6BD, 0)
```

The first 64-bit number represents the SAS address of the target SAS device.

The second number is the boot LUN of the target SAS device.

The third number is the Relative Target Port (RTP)

- SATA Device connected directly to a HBA port.

```
PciRoot(0)/PCI(1,0)/Sas(0x31000004CF13F6BD)
```

The first number represents either a real SAS address reserved by the HBA for above connections, or a fake but unique SAS address generated by the HBA to represent the SATA device.

10.3.4.18.1 Device and Topology Information

First Byte (At offset 40 into the structure):

Bits 0:3:

Value 0x0 -> No Additional Information about device topology.

Value 0x1 -> More Information about device topology valid in this byte.

Value 0x2 -> More Information about device topology valid in this and next 1 byte.

Values 0x3 thru 0xF -> Reserved.

Bits 4:5: Device Type (Valid only if the More Information field above is non-zero)

Value 0x0 -> SAS Internal Device

Value 0x1 -> SATA Internal Device

Value 0x2 -> SAS External Device

Value 0x3 -> SATA External Device

Bits 6:7: Topology / Interconnect (Valid only if the More Information field above is non-zero)

Value 0x0 -> Direct Connect (Connected directly with the HBA Port/Phy)

Value 0x1 -> Expander Connect (Connected thru/via one or more Expanders)

Value 0x2 and 0x3 -> Reserved

10.3.4.18.2 Device and Topology Information

Second Byte (At offset 41 into the structure). Valid only if bits 0-3 of More Information in Byte 40 have a value of 2:

Bits 0-7: Internal Drive/Bay Id (Only applicable if Internal Drive is indicated in Device Type)

Value 0x0 thru 0xFF -> Drive 1 thru Drive 256

10.3.4.18.3 Relative Target Port

At offset 42 into the structure:

This two-byte field shall contain the “Relative Target Port” of the target SAS port. Relative Target Port can be obtained by performing an INQUIRY command to VPD page 0x83 in the target. Implementation of RTP is mandatory for SAS targets as defined in Section 10.2.10 of sas1r07 specification (or later).

NOTE: If a LUN is seen thru multiple RTPs in a given target, then the UEFI driver shall create separate device path instances for both paths. RTP in the device path shall distinguish these two device path instantiations.

NOTE: Changing the values of the SAS/SATA device topology information or the RTP fields of the device path will make UEFI think this is a different device.

10.3.4.18.4 Examples Of Correct Device Path Display Format

Case 1: When Additional Information is not Valid or Not Present (Bits 0:3 of Byte 40 have a value of 0)

```
PciRoot(0)/PCI(1,0)/SAS(0x31000004CF13F6BD, 0)
```

Case 2: When Additional Information is Valid and present (Bits 0:3 of Byte 40 have a value of 1 or 2)

- If Bits 4-5 of Byte 40 (Device and Topology information) indicate an SAS device (Internal or External) i.e., has values 0x0 or 0x2, then the following format shall be used.

```
PciRoot(0)/PCI(1,0)/SAS(0x31000004CF13F6BD, 0, SAS)
```

- If Bits 4-5 of Byte 40 (Device and Topology information) indicate a SATA device (Internal or External) i.e., has a value of 0x1 or 0x3, then the following format shall be used.

```
ACPI(PnP)/PCI(1,0)/SAS(0x31000004CF13F6BD, SATA)
```

10.3.4.19 Serial Attached SCSI (SAS) Extended Device Path

This section defines the extended device node for Serial Attached SCSI (SAS) devices. In this device path the SAS Address and LUN are now defined as arrays to remove the need to endian swap the values.

Table 10.36: SAS Extended Messaging Device Path Structure

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type -3 Messaging
Sub Type	1	1	Sub-type 22 SAS Ex
Length	2	2	Length of this Structure. 32 bytes

continues on next page

Table 10.36 – continued from previous page

SAS Address	4	8	8-byte array of the SAS Address for Serial Attached SCSI Target Port.
Logical Unit Number	20	8	8-byte array of the SAS Logical Unit Number.
SAS/SATA device and Topology Info	28	2	More Information about the device and its interconnect
Relative Target Port	30	2	Relative Target Port (RTP)

The SAS Ex device path clarifies the definition of the Logical Unit Number field to conform with the T-10 SCSI Architecture Model 4 specification. The 8 byte Logical Unit Number field in the device path must conform with a logical unit number returned by a SCSI REPORT LUNS command.

When the SAS Device Path Ex is used with the Extended SCSI Pass Thru Protocol, the UINT64 LUN must be converted to the eight byte array Logical Unit Number field in the device path by treating the eight byte array as an EFI UINT64. For example, a Logical Unit Number array of { 0,1,2,3,4,5,6,7 } becomes a UINT64 of 0x0706050403020100.

When an application client displays or otherwise makes a 64-bit LUN (8 byte array) visible to a user, it should be done in conformance with SAM-4. SAM-4 requires a LUN to be displayed in hexadecimal format with byte 0 first (i.e., on the left) and byte 7 last (i.e., on the right) regardless of the internal representation of the LUN. UEFI defines all data structures a “little endian” and SCSI defines all data structures as “big endian”.

10.3.4.20 iSCSI Device Path

Table 10.37: iSCSI Device Path Node (Base Information)

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 19 - (iSCSI)
Length	2	2	Length of this structure in bytes. Length is (18 + n) bytes
Protocol	4	2	Network Protocol (0 = TCP, 1+ = reserved)
Options	6	2	iSCSI Login Options
Logical Unit Number	8	8	8 byte array containing the iSCSI Logical Unit Number
Target Portal group	16	2	iSCSI Target Portal group tag the initiator intends to establish a session with.
iSCSI Target Name	18	n	iSCSI NodeTarget Name. The length of the name is determined by subtracting the offset of this field from Length.

10.3.4.20.1 iSCSI Login Options

The iSCSI Device Node Options describe the iSCSI login options for the key values:

Bits 0:1:

0 = No Header Digest

2 = Header Digest Using CRC32C

Bits 2-3:

0 = No Data Digest

2 = Data Digest Using CRC32C

Bits 4-9:

Reserved for future use

Bits 10-11:

0 = AuthMethod_CHAP

2 = AuthMethod_None

Bit 12:

0 = CHAP_BI

1 = CHAP_UNI

For each specific login key, none, some or all of the defined values may be configured. If none of the options are defined for a specific key, the iSCSI driver shall propose “None” as the value. If more than one option is configured for a specific key, all the configured values will be proposed (ordering of the values is implementation dependent).

- Portal Group Tag: defines the iSCSI portal group the initiator intends to establish Session with.
- Logical Unit Number: defines the 8 byte SCSI LUN. The Logical Unit Number field must conform to the T-10 SCSI Architecture Model 4 specification. The 8 byte Logical Unit Number field in the device path must conform with a logical unit number returned by a SCSI REPORT LUNS command.
- iSCSI Target Name: defines the iSCSI Target Name for the iSCSI Node. The size of the iSCSI Target Name can be up to a maximum of 223 bytes.

10.3.4.20.2 Device Path Examples

Some examples for the Device Path for the case the boot device connected to iSCSI bootable controller:

- With IPv4 configuration:

```
PciRoot(0)/Pci(19|0)/Mac( 001320F5FA77,0x01)/
IPv4(192.168.0.100,TCP,Static,192.168.0.1)/ iSCSI(iqn.1991-
05.com.microsoft:iscsitarget-iscsidisk-target,0x1,0x0,None,None,None,TCP)/
HD(1,GPT,15E39A00-1DD2-1000-8D7F-00A0C92408FC,0x22,0x27100000)
```

Table 10.38: IPv4 configuration

Byte Offset	Byte Length	Data	Description
0x00	1	0x02	Generic Device Path Header - Type ACPI Device Path
0x01	1	0x01	Sub type - ACPI Device Path
0x02	2	0x0C	Length - 0x0C bytes
0x04	4	0x41D0, 0x0A03	_HID PNP0A03 - 0x41D0 represents the compressed string ‘PNP’ and is encoded in the low order bytes. The compression method is described in the ACPI Specification.
0x08	4	0x0000	_UID
0x0C	1	0x01	Generic Device Path Header - Type Hardware Device Path
0x0D	1	0x01	Sub type - PCI
0x0E	2	0x06	Length - 0x06 bytes
0x10	1	0x0	PCI Function

continues on next page

[illegible]

Table 10.38 – continued from previous page

0x53	1	0x13	Sub type - iSCSI
0x54	2	0x49	Length - 0x49
0x56	2	0x00	Network Protocol
0x58	2	0x800	iSCSI Login Options - AuthMethod_None
0x5A	8		iSCSI LUN
		0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00	
0x62	2	0x01	Target Portal group tag
0x64	55	0x69, 0x71, 0x6E, 0x2E, 0x31, 0x39, 0x39, 0x31, 0x2D, 0x30, 0x35, 0x2E, 0x63, 0x6F, 0x6D, 0x2E, 0x6D, 0x69, 0x63, 0x72, 0x6F, 0x73, 0x6F, 0x66, 0x74,	iSCSI node name.
0x64 (cont.)	55 (cont.)	0x3A, 0x69, 0x73, 0x63, 0x73, 0x69, 0x74, 0x61, 0x72, 0x67, 0x65, 0x74, 0x2D, 0x69, 0x73, 0x63, 0x73, 0x69, 0x64, 0x69, 0x73, 0x6B, 0x2D, 0x74, 0x61, 0x72, 0x67, 0x65, 0x74, 0x00	iSCSI node name (cont.)
0x9B	1	0x04	Generic Device Path Header - Media Device Path
0x9C	1	0x01	Sub type - Hard Drive
0x9D	2	0x2A	Length - 0x2a
0x9F	4	0x1	Partition Number
0xA3	8	0x22	Partition Start
0xAB	8	0x2710000	Partition Size

continues on next page

Table 10.38 – continued from previous page

0xB3	16	Partition Signature	
		0x00, 0x9A, 0xE3, 0x15, 0xD2, 0x1D, 0x00, 0x10, 0x8D, 0x7F, 0x00, 0xA0, 0xC9, 0x24, 0x08, xFC	
0xC3	1	0x02	Partition Format - GPT
0xC4	1	0x02	Signature Type - GUID
0xC5	1	0x7F	Generic Device Path Header - Type End of Hardware Device Path
0xC6	1	0xFF	Sub type - End of Entire Device Path
0xC7	2	0x04	Length - 0x04 bytes

- With IPv6 configuration:

```
PciRoot(0x0)/Pci(0x1C,0x2)/Pci(0x0,0x0)/MAC(001517215593,0x0)/
IPv6(2001:4898:000A:1005:95A6:EE6C:BED3:4859,TCPDHCP,2001:4898:000A:1005:0215
:17FF:FE21:5593)/iSCSI(iqn.1991-05.com.microsoft:iscsiipv6-ipv6test-
target,0x1,0x0,None,None,None,TCP)/HD(1,MBR,0xA0021243,0x800,0x2EE000)
```

Table 10.39: IPv6 configuration

Byte Offset	Byte Length	Data	Description
0x00	1	0x02	Generic Device Path Header - Type ACPI Device Path
0x01	1	0x01	Sub type - ACPI Device Path
0x02	2	0x0C	Length - 0x0C bytes
0x04	4	0x41D0, 0x0A03	_HID PNP0A03 - 0x41D0 represents the compressed string 'PNP' and is encoded in the low order bytes. The compression method is described in the ACPI Specification.
0x08	4	0x0000	_UID
0x0C	1	0x01	Generic Device Path Header - Type Hardware Device Path
0x0D	1	0x01	Sub type - PCI
0x0E	2	0x06	Length - 0x06 bytes
0x10	1	0x02	PCI Function
0x11	1	0x1C	PCI Device
0x12	1	0x01	Generic Device Path Header - Type Hardware Device Path
0x13	1	0x01	Sub type - PCI
0x14	2	0x06	Length - 0x06 bytes

continues on next page

0x16	1	0x00	PCI Function
0x17	1	0x00	PCI Device
0x18	1	0x03	Generic Device Path Header - Messaging Device Path
0x19	1	0x0B	Sub type - MAC Address Device path
0x1A	2	0x25	Length - 0x25
0x1C	32		MAC address for a network interface padded with zeros
		0x00, 0x15,	
		0x17, 0x21,	
		0x55, 0x93,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
0x3C	1	0x01	Network Interface Type - other
0x3D	1	0x03	Generic Device Path Header - Messaging Device Path
0x3E	1	0x0C	Sub type - IPv6
0x3F	2	0x3C	Length - 0x3C

Table 10.39 – continued from previous page

0x41	16	Local IPv6 address - 2001:4898:000A:1005:0215:17FF:FE21:5593	
		0x20, 0x01, 0x48, 0x98, 0x00, 0x0A, 0x10, 0x05, 0x02, 0x15, 0x17, 0xFF, 0xFE, 0x21, 0x55, 0x93	
0x51	16	Remote IPv6 address - 2001:4898:000A:1005:95A6:EE6C:BED3:4859	
		0x20, 0x01, 0x48, 0x98, 0x00, 0x0A, 0x10, 0x05, 0x95, 0xA6, 0xEE, 0x6C, 0xBE, 0xD3, 0x48, 0x59	
0x61	2	0x0000	Local Port Number - 0
0x63	2	0x0CBC	Remote Port Number - 3260
0x65	2	0x6	Network Protocol. See RFC 3232. TCP
0x66	1	1	IP Address Origin
0x67	1		The Prefix Length
0x68	16		The Gateway IP Address
0x78	1	0x03	Generic Device Path Header - Messaging Device Path
0x79	1	0x13	Sub type - iSCSI
0x7A	2	0x46	Length - 0x46
0x7C	2	0x00	Network Protocol
0x7E	2	0x800	iSCSI Login Options - AuthMethod_None
0x81	8		iSCSI LUN
		0x00 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00	
0x89	2	0x01	Target Portal group tag

continues on next page

Table 10.39 – continued from previous page

0x8B	52	iSCSI node name.	
		0x69, 0x71,	
		0x6E, 0x2E,	
		0x31, 0x39,	
		0x39, 0x31,	
		0x2D, 0x30,	
		0x35, 0x2E,	
		0x63, 0x6F,	
		0x6D, 0x2E,	
		0x6D, 0x69,	
		0x63, 0x72,	
		0x6F, 0x73,	
		0x6F, 0x66,	
		0x74, 0x3A,	
		0x69, 0x73,	
		0x63, 0x73,	
		0x69, 0x69,	
		0x70, 0x76,	
0x8B (cont.)	52 (cont.)	iSCSI node name (cont.)	
		0x36, 0x2D,	
		0x69, 0x70,	
		0x76, 0x36,	
		0x74, 0x65,	
		0x73, 0x74,	
		0x2D, 0x74,	
		0x61, 0x72,	
		0x67, 0x65,	
		0x74, 0x00	
0xBF	1	0x04	Generic Device Path Header - Media Device Path
0xC0	1	0x01	Sub type - Hard Drive
0xC1	2	0x2A	Length - 0x2a
0xC3	4	0x1	Partition Number
0xC7	8	0x800	Partition Start
0xCF	8	0x2EE000	Partition Size

continues on next page

Table 10.39 – continued from previous page

0xDF	16	Partition Signature	
		0x43, 0x12,	
		0x02, 0xA0,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
		0x00,	
0xEF	1	0x01	Partition Format - MBR
0xF0	1	0x01	Signature Type - 32bit signature
0xF1	1	0x7F	Generic Device Path Header - Type End of Hardware Device Path
0xF2	1	0xFF	Sub type - End of Entire Device Path
0xF3	2	0x04	Length - 0x04 bytes

10.3.4.21 NVM Express namespace messaging device path node

Table 10.40: NVM Express Namespace Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub Type 23 - NVM Express Namespace
Length	2	2	Length of this structure in bytes. Length is 16 bytes.
Namespace Identifier	4	4	Namespace identifier (NSID). The values of 0 and 0xFFFFFFFF are invalid.
IEEE Extended Unique Identifier	8	8	This field contains the IEEE Extended Unique Identifier (EUI-64). Devices without an EUI-64 value must initialize this field with a value of 0.

Refer to the latest NVM Express® Base Specification for descriptions of Namespace Identifier (NSID) and IEEE Extended Unique Identifier (EUI-64): See “Links to UEFI-Related Documents” (<http://www.nvmexpress.org/index>) under the heading “NVM Express® Specifications”.

Note

When an application client displays or otherwise makes the EUI-64 identifiers visible to a user, the values should be displayed in hexadecimal format with byte 7 first (i.e., on the left) and byte 0 last (i.e., on the right) regardless of the internal representation of the EUI-64.

10.3.4.22 Uniform Resource Identifiers (URI) Device Path

Refer to RFC 3986 for details on the URI contents.

Table 10.41: URI Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub Type 24 - Universal Resource Identifier (URI) Device Path
Length	2	2	Length of this structure in bytes. Length is 4 + n bytes.
...	4	n	Instance of the URI pursuant to RFC 3986. For an empty URI, Length is 4 bytes.

10.3.4.23 UFS (Universal Flash Storage) device messaging devicepath node

Table 10.42: UFS Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 25 - UFS
Length	2	2	Length of this structure in bytes. Length is 6 bytes.
Target ID	4	1	Target ID on the UFS interface (PUN).
LUN	5	1	Logical Unit Number (LUN).

Refer to the UFS 2.0 specification for additional LUN descriptions: See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “UFS 2.0 Specification”.

- *PUN field* : According to current available UFS 2.0 spec, the topology is one device per UFS port. A topology to support multiple devices on a single interface is planned for future revision. So suggest to reserve/introduce this field to support multiple devices per UFS port. This value should be 0 for current UFS2.0 spec compliance.
- *LUN field* : This field is used to specify up to 8 normal LUNs(0-7) and 4 well-known LUNs(81h, D0h, B0h and C4h). For those well-known LUNs, the BIT7 is set. See Figure 10.2 of UFS 2.0 spec for details.

10.3.4.24 SD (Secure Digital) Device Path

Table 10.43: SD Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 26 - SD
Length	2	2	Length of this structure in bytes. Length is 5 bytes.
Slot Number	4	1	Slot Number

10.3.4.25 EFI Bluetooth Device Path

Table 10.44: EFI Bluetooth Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 27 - Bluetooth
Length	2	2	Length of this structure in bytes. Length is 10 bytes.
Bluetooth Device Address	4	6	48-bit Bluetooth device address.

10.3.4.26 Wireless Device Path

Table 10.45: Wi-Fi Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 28 - Wi-Fi Device Path
Length	2	2	Length of this structure in bytes. Length is 36 bytes.
SSID	4	32	SSID in octet string

10.3.4.27 eMMC (Embedded Multi-Media Card) Device Path

Table 10.46: eMMC Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 29 - eMMC
Length	2	2	Length of this structure in bytes. Length is 5 bytes.
Slot Number	4	1	Slot Number

10.3.4.28 EFI BluetoothLE Device Path

Table 10.47: EFI BluetoothLE Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 30 - BluetoothLE
Length	2	2	Length of this structure in bytes. Length is 11 bytes.
Bluetooth Device Address	4	6	48-bit Bluetooth device address
Address Type	10	1	0x00 - Public Device Address 0x01 - Random Device Address

10.3.4.29 DNS Device Path

Table 10.48: DNS Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device Path
Sub-Type	1	1	Sub-Type 31 - DNS Device Path
Length	2	2	Length of this structure in bytes. Length is 5 + n bytes.
IsIPv6	4	1	0x00 - The DNS server address is IPv4 address. 0x01 - The DNS server address is IPv6 address.
...	5	n	One or more instances of the DNS server address in EFI_IP_ADDRESS.

10.3.4.30 NVDIMM Namespace path

This device path describes a bootable NVDIMM namespace that is defined by a namespace label. The presence of this type of device path indicates that UEFI supports booting to the namespace including any address abstraction specified by the namespace label. Refer to the NVDIMM Label Protocol section to retrieve relevant details about the namespace.

Table 10.49: NVDIMM Namespace Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 3 - Messaging Device
Sub-Type	1	1	Sub-type 32 - NVDIMM Namespace
Length	2	2	20 - Single namespace UUID is supported.
Uuid	4	16	Namespace unique label identifier UUID. See the Uuid description in the NVDIMM Label Protocol - Label definitions section for details on this field.

10.3.4.31 REST Service Device Path

Table 10.50: REST Service Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 0x03 - Messaging Device Path
Sub-Type	1	1	Sub-Type 33- REST Service Device Path
Length	2	2	Length of this structure in bytes. Length is 6 bytes.
REST Service	4	1	0x01 = Redfish REST Service - 0x02 = OData REST Service
Access Mode	5	1	(0x01) In-Band REST Service, - (0x02) Out-of-band REST Service.

Device path example of Out-of-band Redfish REST Service through NIC:

```
PciRoot(0x2)/Pci(0x2,0x0)/Pci(0x0,0x0)/MAC(FD19FA100672,0x0)/
IPv4(0.0.0.0,0x0,DHCP,0.0.0.0,0.0.0.0,0.0.0.0)/RestService(1,2)
```

Table 10.51: Vendor-Specific REST Service Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 0x03 - Messaging Device Path
Sub-Type	1	1	Sub-Type 33- REST Service Device Path
Length	2	2	Length of this structure in bytes. Length is 21 + n bytes.
REST Service	4	1	0xFF = Vendor specific REST Service
Access Mode	5	1	(0x01) In-Band REST Service, (0x02) Out-of-band REST Service.
Vendor specific REST service GUID	6	16	GUID of vendor specific REST service.
Vendor defined data	22	n	Vendor-defined data.

Device path example of In-band vendor-specific REST Service through BMC:

```
PciRoot(0x2)/Pci(0x2,0x0)/Pci(0x0,0x0)/BMC(0,0xf0000000)/RestService(0xff, 1,
00000000-0000-0000-0000000000000000,0,0)
```

10.3.4.32 NVMe over Fabric (NVMe-oF) Namespace Device Path

This device path describes a bootable NVMe-oF™ namespace that is defined by a unique Namespace and Subsystem NQN identity.

Table 10.52: NVMe over Fabric (NVMe-oF) Namespace Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	00	1	Type 3 – Messaging Device Path
Sub-Type	01	1	Sub-Type 34 - NVMe-oF Namespace Device Path
Length	02	2	Length of this Structure in bytes. Length is 20+n bytes where n is the length of the SubsystemNQN
NIDT	04	1	Namespace Identifier Type (NIDT), for globally unique type values defined in the CNS 03h NIDT field (1h, 2h, or 3h) by the NVM Express® Base Specification®.
NID	05	16	Namespace Identifier (NID), a globally unique value defined in the Namespace Identification Descriptor list (CNS 03h) by the NVM Express® Base Specification in big endian format.
Subsystem-NQN	21	n	Unique identifier of an NVM subsystem stored as a null-terminated UTF-8 string of n-bytes in compliance with the NVMe Qualified Name in the NVM Express® Base Specification. Subsystem NQN is used for purposes of identification and authentication. Maximum length of 224 bytes.

Refer to the latest NVM Express® Base Specification for descriptions of Subsystem NQN and Namespace Identifier: See “Links to UEFI-Related Documents” (<http://uefi.org/uefi>) under the heading “NVM Express® Specifications.”

10.3.4.33 NVMe over Fabric (NVMe-oF) Namespace Device Path Example

Device path example of NVMe-oF connection to a NVM Subsystem with IPv4 configuration and NSUUID (RFC 4122) as NID:

```
PciRoot(0)/Pci(19|0)/Mac(001320F5FA77,0x01)/IPv4(192.168.0.100,TCP,Static,192.168.0.1)/
NVMeoF(nqn.1991-05.org.uefi:nvmeoftarget-nvmeofdisk-target,urn:uuid:4eff7f8e-d353-4e9b-
a4ec-deea8eab84d7)/
HD(1,GPT,15E39A00-1DD2-1000-8D7F-00A0C92408FC,0x22,0x2710000)
```

Table 10.53: NVMe over Fabric (NVMe-oF) Namespace Device Path Example

Byte Offset	Byte Length	Data	Description
0x00	1	0x02	Generic Device Path Header – Type ACPI Device Path
0x01	1	0x01	Sub-Type – ACPI Device Path
0x02	2	0x0C	Length – 12 bytes
0x04	4	0x41D0, 0x0A03	_HID PNP0A03 – 0x41D0 represents the compressed string ‘PNP’ and is encoded in the low order bytes. The compression method is described in the ACPI Specification.
0x08	4	0x0000	_UID
0x0C	1	0x01	Generic Device Path Header – Type Hardware Device Path
0x0D	1	0x01	Sub-Type – PCI
0x0E	2	0x06	Length – 6 bytes
0x10	1	0x00	PCI Function
0x11	1	0x19	PCI Device
0x12	1	0x03	Generic Device Path Header – Messaging Device Path
0x13	1	0x0B	Sub-Type – MAC Address Device path
0x14	2	0x25	Length – 37 bytes
0x16	32	0x00, 0x13, 0x20, 0xF5, 0xFA, 0x77, 0x00	MAC address for a network interface padded with zeros
0x36	1	0x01	Network Interface Type - other
0x37	1	0x03	Generic Device Path Header – Messaging Device Path
0x38	1	0x0C	Sub-Type – IPv4
0x39	2	0x1B	Length – 27 bytes
0x3b	4	0xC0, 0xA8, 0x00, 0x01	Local IPv4 address – 192.168.0.1
0x3F	4	0xC0, 0xA8, 0x00, 0x64	Remote IPv4 address – 192.168.0.100
0x43	2	0x0000	Local Port Number – 0
0x45	2	0x0CBC	Remote Port Number – 3260
0x47	2	0x06	Network Protocol. See RFC 3232. TCP
0x49	1	1	Static IP Address

continues on next page

Table 10.53 – continued from previous page

0x4A	4	Gateway IP Address	
0x4E	4	Subnet mask	
0x52	1	0x03	Generic Device Path Header – Messaging Device Path
0x53	1	0x34	Sub-Type 34 - NVMe-oF Device Path
0x54	2	0x48	Length – 72 bytes
0x56	1	0x03	Namespace Identifier Type (NIDT) – 03h – Namespace UUID
0x57	16	0x4E, 0xFF, 0x7F, 0x8E, 0xD3, 0x53, 0x4E, 0x9B, 0xA4, 0xEC, 0xDE, 0xEA, 0x8E, 0xAB, 0x84, 0xD7	Namespace Identifier (NID) - as per RFC 4122
0x67	52	0x6E, 0x71, 0x6E, 0x2E, 0x31, 0x39, 0x39, 0x31, 0x2D, 0x30, 0x35, 0x2E, 0x6F, 0x72, 0x67, 0x2E, 0x75, 0x65, 0x66, 0x69, 0x3A, 0x6E, 0x76, 0x6D, 0x65, 0x6F, 0x66, 0x74, 0x61, 0x72, 0x67, 0x65, 0x74, 0x2D, 0x6E, 0x76, 0x6D, 0x65, 0x6F, 0x66, 0x64, 0x69, 0x73, 0x6B, 0x2D, 0x74, 0x61, 0x72, 0x67, 0x65, 0x74, 0x00	SubsystemNQN
0x9B	1	0x04	Generic Device Path Header – Media Device Path
0x9C	1	0x01	Sub-Type – Hard Drive
0x9D	2	0x2A	Length – 42 bytes
0x9F	4	0x01	Partition Number
0xA3	8	0x22	Partition Start
0xAB	8	0x2710000	Partition Size
0xB3	16	0x00, 0x9A, 0xE3, 0x15, 0xD2, 0x1D, 0x00, 0x10, 0x8D, 0x7F, 0x00, 0xA0, 0xC9, 0x24, 0x08, 0xFC	Partition Signature
0xC3	1	0x02	Partition Format – GPT
0xC4	1	0x02	Signature Type – GUID
0xC5	1	0xFF	Generic Device Path Header – End of Hardware Device Path
0xC6	1	0xFF	Sub-Type – End of Entire Device Path
0xC7	2	0x04	Length – 4 bytes

10.3.5 Media Device Path

This Device Path is used to describe the portion of the medium that is being abstracted by a boot service. An example of Media Device Path would be defining which partition on a hard drive was being used.

10.3.5.1 Hard Drive

The Hard Drive Media Device Path is used to represent a partition on a hard drive. Each partition has at least Hard Drive Device Path node, each describing an entry in a partition table. EFI supports MBR and GPT partitioning formats. Partitions are numbered according to their entry in their respective partition table, starting with 1. Partitions are addressed in EFI starting at LBA zero. A partition number of zero can be used to represent the raw hard drive or a raw extended partition.

The partition format is stored in the Device Path to allow new partition formats to be supported in the future. The Hard Drive Device Path also contains a Disk Signature and a Disk Signature Type. The disk signature is maintained by the OS and only used by EFI to partition Device Path nodes. The disk signature enables the OS to find disks even after they have been physically moved in a system.

Load Option Processing defines special rules for processing the Hard Drive Media Device Path. These special rules enable a disk's location to change and still have the system boot from the disk.

Table 10.54: Hard Drive Media Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 4 — Media Device Path
Sub-Type	1	1	Sub-Type 1 — Hard Drive
Length	2	2	Length of this structure in bytes. Length is 42 bytes.
Partition Number	4	4	Describes the entry in a partition table, starting with entry 1. Partition number zero represents the entire device. Valid partition numbers for a MBR partition are [1, 4]. Valid partition numbers for a GPT partition are [1, NumberOfPartitionEntries].
Partition Start	8	8	Starting LBA of the partition on the hard drive
Partition Size	16	8	Size of the partition in units of Logical Blocks
Partition Signature	24	16	Signature unique to this partition: If SignatureType is 0, this field has to be initialized with 16 zeroes. If SignatureType is 1, the MBR signature is stored in the first 4 bytes of this field. The other 12 bytes are initialized with zeroes. If SignatureType is 2, this field contains a 16 byte signature.
Partition Format	40	1	Partition Format: (Unused values reserved) 0x01 - PC-AT compatible legacy MBR (<i>Legacy MBR</i>). Partition Start and Partition Size come from <i>PartitionStartingLBA</i> and <i>PartitionSizeInLBA</i> for the partition. 0x02—GUID Partition Table
Signature Type	41	1	Type of Disk Signature: (Unused values reserved) 0x00 - No Disk Signature. 0x01 - 32-bit signature from address 0x1b8 of the type 0x01 MBR. 0x02 - GUID signature.

10.3.5.2 CD-ROM Media Device Path

The CD-ROM Media Device Path is used to define a system partition that exists on a CD-ROM. The CD-ROM is assumed to contain an ISO-9660 file system and follow the CD-ROM “El Torito” format. The Boot Entry number from the Boot Catalog is how the “El Torito” specification defines the existence of bootable entities on a CD-ROM. In EFI the bootable entity is an EFI System Partition that is pointed to by the Boot Entry.

Table 10.55: CD-ROM Media Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 4 - Media Device Path.
Sub-Type	1	1	Sub-Type 2 - CD-ROM “El Torito” Format.
Length	2	2	Length of this structure in bytes. Length is 24 bytes.
Boot Entry	4	4	Boot Entry number from the Boot Catalog. The Initial/Default entry is defined as zero.
Partition Start	8	8	Starting RBA of the partition on the medium. CD-ROMs use Relative logical Block Addressing.
Partition Size	16	8	Size of the partition in units of Blocks, also called Sectors.

10.3.5.3 Vendor-Defined Media Device Path

Table 10.56: Vendor-Defined Media Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 4 - Media Device Path.
Sub-Type	1	1	Sub-Type 3 - Vendor.
Length	2	2	Length of this structure in bytes. Length is 20 + n bytes.
Vendor GUID	4	16	Vendor-assigned GUID that defines the data that follows.
Vendor Defined Data	20	n	Vendor-defined variable size data.

10.3.5.4 File Path Media Device Path

Table 10.57: File Path Media Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 4 - Media Device Path.
Sub-Type	1	1	Sub-Type 4 - File Path.
Length	2	2	Length of this structure in bytes. Length is 4 + n bytes.

continues on next page

Table 10.57 – continued from previous page

Path Name	4	<i>N</i>	A NULL-terminated Path string including directory and file names. The length of this string <i>n</i> can be determined by subtracting 4 from the Length entry. A device path may contain one or more of these nodes. Each node can optionally add a “” separator to the beginning and/or the end of the Path Name string. The complete path to a file can be found by logically concatenating all the Path Name strings in the File Path Media Device Path nodes. This is typically used to describe the directory path in one node, and the filename in another node.
-----------	---	----------	--

Rules for Path Name conversion:

- When concatenating two Path Names, ensure that the resulting string does not contain a double-separator “\”. If it does, convert that double-separator to a single-separator.
- In the case where a Path Name which has no end separator is being concatenated to a Path Name with no beginning separator, a separator will need to be inserted between the Path Names.
- Single file path nodes with no directory path data are presumed to have their files located in the root directory of the device.

10.3.5.5 Media Protocol Device Path

The Media Protocol Device Path is used to denote the protocol that is being used in a device path at the location of the path specified. Many protocols are inherent to the style of device path.

Table 10.58: Media Protocol Media Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 4 - Media Device Path.
Sub-Type	1	1	Sub-Type 5 - Media Protocol.
Length	2	2	Length of this structure in bytes. Length is 20 bytes.
Protocol GUID	4	16	The ID of the protocol.

10.3.5.6 PIWG Firmware File

This type is used by systems implementing the UEFI PI Specification to describe a firmware file. The exact format and usage are defined in that specification.

Table 10.59: PIWG Firmware File Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 4 - Media Device Path.
Sub-Type	1	1	Sub-Type 6 - PIWG Firmware File.
Length	2	2	Length of this structure in bytes. Length is 4 + <i>n</i> bytes.
...	4	<i>n</i>	Contents are defined in the UEFI PI Specification.

10.3.5.7 PIWG Firmware Volume

This type is used by systems implementing the UEFI PI Specification to describe a firmware volume. The exact format and usage are defined in that specification.

Table 10.60: PIWG Firmware Volume Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 4 - Media Device Path.
Sub-Type	1	1	Sub-Type 7 - PIWG Firmware Volume.
Length	2	2	Length of this structure in bytes. Length is 4 + n bytes.
...	4	n	Contents are defined in the UEFI PI Specification.

10.3.5.8 Relative Offset Range

This device path node specifies a range of offsets relative to the first byte available on the device. The starting offset is the first byte of the range and the ending offset is the last byte of the range (not the last byte + 1).

Table 10.61: Relative Offset Range

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 4 - Media Device Path
Sub-Type	1	1	Sub-Type 8 - Relative Offset Range
Length	2	2	Length of this structure in bytes.
Reserved	4	4	Reserved for future use.
Starting Offset	8	8	Offset of the first byte, relative to the parent device node.
Ending Offset	16	8	Offset of the last byte, relative to the parent device node.

10.3.5.9 RAM Disk

Table 10.62: RAM Disk Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 4 - Media Device Path
Sub-Type	1	1	Sub Type 9 - RAM Disk Device Path
Length	2	2	Length of this structure in bytes. Length is 38 bytes.
Starting Address	4	8	Starting Memory Address.
Ending Address	12	8	Ending Memory Address.
Disk Type GUID	20	16	GUID that defines the type of the RAM Disk. The GUID can be any of the values defined below, or a vendor defined GUID.
Disk Instance	36	2	RAM Disk instance number, if supported. The default value is zero.

The following GUIDs are used with a RAM Disk Device Path to describe the RAM Disk Type. Additional GUIDs can be generated to describe additional RAM Disk Types. The Disk Type GUID values used in the RAM Disk device path must match the corresponding values in the Address Range Type GUID of the ACPI NFIT table. Refer to the ACPI specification for details.

This GUID defines a RAM Disk supporting a raw disk format in volatile memory:

```
#define EFI_VIRTUAL_DISK_GUID \
{ 0x77AB535A, 0x45FC, 0x624B, \
{0x55, 0x60, 0xF7, 0xB2, 0x81, 0xD1, 0xF9, 0x6E } }
```

This GUID defines a RAM Disk supporting an ISO image in volatile memory:

```
#define EFI_VIRTUAL_CD_GUID \
{ 0x3D5ABD30, 0x4175, 0x87CE, \
{0x6D, 0x64, 0xD2, 0xAD, 0xE5, 0x23, 0xC4, 0xBB } }
```

This GUID defines a RAM Disk supporting a raw disk format in persistent memory:

```
#define EFI_PERSISTENT_VIRTUAL_DISK_GUID \
{ 0x5CEA02C9, 0x4D07, 0x69D3, \
{0x26, 0x9F, 0x44, 0x96, 0xFB, 0xE0, 0x96, 0xF9 } }
```

This GUID defines a RAM Disk supporting an ISO image in persistent memory:

```
#define EFI_PERSISTENT_VIRTUAL_CD_GUID \
{ 0x08018188, 0x42CD, 0xBB48, \
{0x10, 0x0F, 0x53, 0x87, 0xD5, 0x3D, 0xED, 0x3D } }
```

10.3.6 BIOS Boot Specification Device Path

This Device Path is used to describe the booting of non-EFI-aware operating systems. This Device Path is based on the IPL and BCV table entry data structures defined in Appendix A of the *BIOS Boot Specification*. The BIOS Boot Specification Device Path defines a complete Device Path and is not used with other Device Path entries. This Device Path is only needed to enable platform firmware to select a legacy non-EFI OS as a boot option.

Table 10.63: BIOS Boot Specification Device Path

Mnemonic	Byte Offset	Byte Length	Description
Type	0	1	Type 5 - BIOS Boot Specification Device Path.
Sub-Type	1	1	Sub-Type 1 - BIOS Boot Specification Version 1.01.
Length	2	2	Length of this structure in bytes. Length is 8 + n bytes.
Device Type	4	2	Device Type as defined by the BIOS Boot Specification.
Status Flag	6	2	Status Flags as defined by the BIOS Boot Specification
Description String	8	n	A null-terminated ASCII string that describes the boot device to a user. The size of this string <i>n</i> can be determined by subtracting 8 from the Length entry.

Example BIOS Boot Specification Device Types include:

- 00h = Reserved
- 01h = Floppy
- 02h = Hard Disk
- 03h = CD-ROM
- 04h = PCMCIA
- 05h = USB

- 06h = Embedded network
- 07h..7Fh = Reserved
- 80h = BEV device
- 81h..FEh = Reserved
- FFh = Unknown

NOTE: When UEFI Secure Boot is enabled, attempts to boot non-UEFI OS shall fail; *Firmware/OS Key Exchange: Passing Public Keys* .

10.4 Device Path Generation Rules

10.4.1 Housekeeping Rules

The Device Path is a set of Device Path nodes. The Device Path must be terminated by an End of Device Path node with a sub-type of End the Entire Device Path. A NULL Device Path consists of a single End Device Path Node. A Device Path that contains a NULL pointer and no Device Path structures is illegal.

All Device Path nodes start with the generic Device Path structure. Unknown Device Path types can be skipped when parsing the Device Path since the length field can be used to find the next Device Path structure in the stream. Any future additions to the Device Path structure types will always start with the current standard header. The size of a Device Path can be determined by traversing the generic Device Path structures in each header and adding up the total size of the Device Path. This size will include the four bytes of the End of Device Path structure.

Multiple hardware devices may be pointed to by a single Device Path. Each hardware device will contain a complete Device Path that is terminated by the Device Path End Structure. The Device Path End Structures that do not end the Device Path contain a sub-type of End This Instance of the Device Path. The last Device Path End Structure contains a sub-type of End Entire Device Path.

10.4.2 Rules with ACPI _HID and _UID

As described in the ACPI specification, ACPI supports several different kinds of device identification objects, including _HID, _CID and _UID. The _UID device identification objects are optional in ACPI and only required if more than one _HID exists with the same ID. The ACPI Device Path structure must contain a zero in the _UID field if the ACPI name space does not implement _UID. The _UID field is a unique serial number that persists across reboots.

If a device in the ACPI name space has a _HID and is described by a _CRS (Current Resource Setting) then it should be described by an ACPI Device Path structure. A _CRS implies that a device not mapped by any other standard. A _CRS is used by ACPI to make a nonstandard device into a Plug and Play device. The configuration methods in the ACPI name space allow the ACPI driver to configure the device in a standard fashion. The presence of a _CID determines whether the ACPI Device Path node or the Expanded ACPI Device Path node should be used.

See Table below.

Table 10.64: ACPI_CRS to EFI Device Path Mapping

ACPI _CRS Item	EFI Device Path
PCI Root Bus	ACPI Device Path: _HID PNP0A03, _UID
Floppy	ACPI Device Path: _HID PNP0604, _UID drive select encoding 0-3
Keyboard	ACPI Device Path: _HID PNP0301, _UID 0
Serial Port	ACPI Device Path: _HID PNP0501, _UID Serial Port COM number 0-3
Parallel Port	ACPI Device Path: _HID PNP0401, _UID LPT number 0-3

Support of root PCI bridges requires special rules in the EFI Device Path. A root PCI bridge is a PCI device usually contained in a chipset that consumes a proprietary bus and produces a PCI bus. In typical desktop and mobile systems there is only one root PCI bridge. On larger server systems there are typically multiple root PCI bridges. The operation of root PCI bridges is not defined in any current PCI specification. A root PCI bridge should not be confused with a PCI to PCI bridge that both consumes and produces a PCI bus. The operation and configuration of PCI to PCI bridges is fully specified in current PCI specifications.

Root PCI bridges will use the plug and play ID of PNP0A03, This will be stored in the ACPI Device Path `_HID` field, or in the Expanded ACPI Device Path `_CID` field to match the ACPI name space. The `_UID` in the ACPI Device Path structure must match the `_UID` in the ACPI name space.

10.4.3 Rules with ACPI `_ADR`

If a device in the ACPI name space can be completely described by a `_ADR` object then it will map to an EFI ACPI, Hardware, or Message Device Path structure. A `_ADR` method implies a bus with a standard enumeration algorithm. If the ACPI device has a `_ADR` and a `_CRS` method, then it should also have a `_HID` method and follow the rules for using `_HID`. See the table below as it relates the ACPI `_ADR` bus definition to the EFI Device Path:

Table 10.65: ACPI `_ADR` to EFI Device Path Mapping

ACPI <code>_ADR</code> Bus	EFI Device Path
EISA	Not supported
Floppy Bus	ACPI Device Path: <code>_HID</code> PNP0604, <code>_UID</code> drive select encoding 0-3
IDE Controller	ATAPI Message Device Path: Maser/Slave : LUN
IDE Channel	ATAPI Message Device Path: Maser/Slave : LUN
PCI	PCI Hardware Device Path
PCMCIA	Not Supported
PC CARD	PC CARD Hardware Device Path
SMBus	Not Supported
SATA bus	SATA Messaging Device Path

10.4.4 Hardware vs. Messaging Device Path Rules

Hardware Device Paths are used to define paths on buses that have a standard enumeration algorithm and that relate directly to the coherency domain of the system. The coherency domain is defined as a global set of resources that is visible to at least one processor in the system. In a typical system this would include the processor memory space, IO space, and PCI configuration space.

Messaging Device Paths are used to define paths on buses that have a standard enumeration algorithm, but are not part of the global coherency domain of the system. SCSI and Fibre Channel are examples of this kind of bus. The Messaging Device Path can also be used to describe virtual connections over network-style devices. An example would be the TCP/IP address of an internet connection.

Thus, Hardware Device Path is used if the bus produces resources that show up in the coherency resource domain of the system. A Message Device Path is used if the bus consumes resources from the coherency domain and produces resources outside the coherency domain of the system.

10.4.5 Media Device Path Rules

The Media Device Path is used to define the location of information on a medium. Hard Drives are subdivided into partitions by the MBR and a Media Device Path is used to define which partition is being used. A CD-ROM has boot partitions that are defined by the “El Torito” specification, and the Media Device Path is used to point to these partitions.

A *EFI_BLOCK_IO_PROTOCOL* is produced for both raw devices and partitions on devices. This allows the *EFI_SIMPLE_FILE_SYSTEM_PROTOCOL* protocol to not have to understand media formats. The *EFI_BLOCK_IO_PROTOCOL* for a partition contains the same Device Path as the parent *EFI_BLOCK_IO_PROTOCOL* for the raw device with the addition of a Media Device Path that defines which partition is being abstracted.

The Media Device Path is also used to define the location of a file in a file system. This Device Path is used to load files and to represent what file an image was loaded from.

10.4.6 Other Rules

The BIOS Boot Specification Device Path is not a typical Device Path. A Device Path containing the BIOS Boot Specification Device Path should only contain the required End Device Path structure and no other Device Path structures. The BIOS Boot Specification Device Path is only used to allow the EFI boot menus to boot a legacy operating system from legacy media.

The EFI Device Path can be extended in a compatible fashion by assigning your own vendor GUID to a Hardware, Messaging, or Media Device Path. This extension is guaranteed to never conflict with future extensions of this specification.

The EFI specification reserves all undefined Device Path types and subtypes. Extension is only permitted using a Vendor GUID Device Path entry.

10.5 Device Path Utilities Protocol

This section describes the *EFI_DEVICE_PATH_UTILITIES_PROTOCOL*, which aids in creating and manipulating device paths.

10.5.1 EFI_DEVICE_PATH_UTILITIES_PROTOCOL

Summary

Creates and manipulates device paths and device nodes.

GUID

```
// {0379BE4E-D706-437d-B037-EDB82FB772A4}
#define EFI_DEVICE_PATH_UTILITIES_PROTOCOL_GUID \
    {0x379be4e, 0xd706, 0x437d, \
     {0xb0, 0x37, 0xed, 0xb8, 0x2f, 0xb7, 0x72, 0xa4} }
```

Protocol Interface Structure

```
typedef struct _EFI_DEVICE_PATH_UTILITIES_PROTOCOL {
    EFI_DEVICE_PATH_UTILS_GET_DEVICE_PATH_SIZE    GetDevicePathSize;
    EFI_DEVICE_PATH_UTILS_DUP_DEVICE_PATH         DuplicateDevicePath;
    EFI_DEVICE_PATH_UTILS_APPEND_PATH              AppendDevicePath;
```

(continues on next page)

(continued from previous page)

EFI_DEVICE_PATH_UTILS_APPEND_NODE	AppendDeviceNode;
EFI_DEVICE_PATH_UTILS_APPEND_INSTANCE	AppendDevicePathInstance;
EFI_DEVICE_PATH_UTILS_GET_NEXT_INSTANCE	GetNextDevicePathInstance;
EFI_DEVICE_PATH_UTILS_IS_MULTI_INSTANCE	IsDevicePathMultiInstance;
EFI_DEVICE_PATH_UTILS_CREATE_NODE	CreateDeviceNode;
} EFI_DEVICE_PATH_UTILITIES_PROTOCOL;	

Parameters

GetDevicePathSize

Returns the size of the specified device path, in bytes.

DuplicateDevicePath

Duplicates a device path structure.

AppendDeviceNode

Appends the device node to the specified device path.

AppendDevicePath

Appends the device path to the specified device path.

AppendDevicePathInstance

Appends a device path instance to another device path.

GetNextDevicePathInstance

Retrieves the next device path instance from a device path data structure.

IsDevicePathMultiInstance

Returns **TRUE** if this is a multi-instance device path.

CreateDeviceNode

Allocates memory for a device node with the specified type and sub-type.

Description

The `EFI_DEVICE_PATH_UTILITIES_PROTOCOL` provides common utilities for creating a manipulating device paths and device nodes.

10.5.2 EFI_DEVICE_PATH_UTILITIES_PROTOCOL.GetDevicePathSize()

Summary

Returns the size of the device path, in bytes.

Prototype

```
typedef
UINTN
(EFIAPI *EFI_DEVICE_PATH_UTILS_GET_DEVICE_PATH_SIZE) (
    IN CONST EFI_DEVICE_PATH_PROTOCOL          *DevicePath
);
```

Parameters

DevicePath

Points to the start of the EFI device path.

Description

This function returns the size of the specified device path, in bytes, including the end-of-path tag. If *DevicePath* is NULL then zero is returned.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in *EFI Device Path Protocol*.

10.5.3 EFI_DEVICE_PATH_UTILITIES_PROTOCOL.DuplicateDevicePath()

Summary

Create a duplicate of the specified path.

Prototype

```
typedef
EFI_DEVICE_PATH_PROTOCOL*
(EFIAPI *EFI_DEVICE_PATH_UTILS_DUP_DEVICE_PATH) (
    IN CONST EFI_DEVICE_PATH_PROTOCOL    *DevicePath
);
```

Parameters

DevicePath

Points to the source device path.

Description

This function creates a duplicate of the specified device path. The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated. If *DevicePath* is NULL then NULL will be returned and no memory will be allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in *EFI Device Path Protocol*.

Returns

This function returns a pointer to the duplicate device path or NULL if there was insufficient memory.

10.5.4 EFI_DEVICE_PATH_UTILITIES_PROTOCOL.AppendDevicePath()

Summary

Create a new path by appending the second device path to the first.

Prototype

```
typedef
EFI_DEVICE_PATH_PROTOCOL*
(EFIAPI *EFI_DEVICE_PATH_UTILS_APPEND_PATH) (
    IN CONST EFI_DEVICE_PATH_PROTOCOL    *Src1 ,
    IN CONST EFI_DEVICE_PATH_PROTOCOL    *Src2
);
```

Parameters

Src1

Points to the first device path.

Src2

Points to the second device path.

Description

This function creates a new device path by appending a copy of the second device path to a copy of the first device path in a newly allocated buffer. Only the end-of-device-path device node from the second device path is retained. If *Src1* is NULL and *Src2* is non-NULL, then a duplicate of *Src2* is returned. If *Src1* is non-NULL and *Src2* is NULL, then a duplicate of *Src1* is returned. If *Src1* and *Src2* are both NULL, then a copy of an end-of-device-path is returned.

The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in *EFI Device Path Protocol*.

Returns

This function returns a pointer to the newly created device path or NULL if memory could not be allocate.

10.5.5 EFI_DEVICE_PATH_UTILITIES_PROTOCOL.AppendDeviceNode()

Summary

Creates a new path by appending the device node to the device path.

Prototype

```
typedef
EFI_DEVICE_PATH_PROTOCOL*
(EFIAPI *EFI_DEVICE_PATH_UTILS_APPEND_NODE) (
IN CONST EFI_DEVICE_PATH_PROTOCOL      *DevicePath,
IN CONST EFI_DEVICE_PATH_PROTOCOL      *DeviceNode
);
```

Parameters

DevicePath

Points to the device path.

DeviceNode

Points to the device node.

Description

This function creates a new device path by appending a copy of the specified device node to a copy of the specified device path in an allocated buffer. The end-of-device-path device node is moved after the end of the appended device node. If *DeviceNode* is NULL then a copy of *DevicePath* is returned. If *DevicePath* is NULL then a copy of *DeviceNode*, followed by an end-of-device path device node is returned. If both *DeviceNode* and *DevicePath* are NULL then a copy of an end-of-device-path device node is returned.

The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in *EFI Device Path Protocol*.

Returns

This function returns a pointer to the allocated device path, or NULL if there was insufficient memory.

10.5.6 EFI_DEVICE_PATH_UTILITIES_PROTOCOL.AppendDevicePathInstance()

Summary

Creates a new path by appending the specified device path instance to the specified device path.

Prototype

```
typedef
EFI_DEVICE_PATH_PROTOCOL*
(EFIAPI *EFI_DEVICE_PATH_UTILS_APPEND_INSTANCE) (
    IN CONST EFI_DEVICE_PATH_PROTOCOL      *DevicePath,
    IN CONST EFI_DEVICE_PATH_PROTOCOL      *DevicePathInstance
);
```

Parameters

DevicePath

Points to the device path. If NULL, then ignored.

DevicePathInstance

Points to the device path instance

Description

This function creates a new device path by appending a copy of the specified device path instance to a copy of the specified device path in an allocated buffer. The end-of-device-path device node is moved after the end of the appended device node and a new end-of-device-path-instance node is inserted between. If *DevicePath* is NULL, then a copy of *DevicePathInstance* is returned instead.

The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in [EFI Device Path Protocol](#).

Returns

This function returns a pointer to the newly created device path or NULL if *DevicePathInstance* is NULL or there was insufficient memory.

10.5.7 EFI_DEVICE_PATH_UTILITIES_PROTOCOL.GetNextDevicePathInstance()

Summary

Creates a copy of the current device path instance and returns pointer to the next device path instance.

Prototype

```
typedef
EFI_DEVICE_PATH_PROTOCOL*
(EFIAPI *EFI_DEVICE_PATH_UTILS_GET_NEXT_INSTANCE) (
    IN OUT EFI_DEVICE_PATH_PROTOCOL      **DevicePathInstance,
    OUT UINTN                            *DevicePathInstanceSize* OPTIONAL
);
```

Parameters

DevicePathInstance

On input, this holds the pointer to the current device path instance. On output, this holds the pointer to the next device path instance or NULL if there are no more device path instances in the device path.

DevicePathInstanceSize

On output, this holds the size of the device path instance, in bytes or zero, if *DevicePathInstance* is NULL. If NULL, then the instance size is not output.

Description

This function creates a copy of the current device path instance. It also updates *DevicePathInstance* to point to the next device path instance in the device path (or NULL if no more) and updates *DevicePathInstanceSize* to hold the size of the device path instance copy.

The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in *EFI Device Path Protocol*.

Returns

This function returns a pointer to the copy of the current device path instance or NULL if *DevicePathInstance* was NULL on entry or there was insufficient memory.

10.5.8 EFI_DEVICE_PATH_UTILITIES_PROTOCOL.CreateDeviceNode()

Summary

Creates a device node

Prototype

```
typedef
EFI_DEVICE_PATH_PROTOCOL*
(EFI_API *EFI_DEVICE_PATH_UTILS_CREATE_NODE) (
    IN UINT8          NodeType,
    IN UINT8          NodeSubType,
    IN UINT16         NodeLength
);
```

Parameters

NodeType

NodeType is the device node type (EFI_DEVICE_PATH_PROTOCOL.Type) for the new device node.

NodeSubType

NodeSubType is the device node sub-type (EFI_DEVICE_PATH_PROTOCOL.SubType) for the new device node.

NodeLength

NodeLength is the length of the device node (EFI_DEVICE_PATH_PROTOCOL.Length) for the new device node.

Description

This function creates a new device node in a newly allocated buffer.

The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in *EFI Device Path Protocol*.

Returns

This function returns a pointer to the created device node or NULL if *NodeLength* is less than the size of the header or there was insufficient memory.

10.5.9 EFI_DEVICE_PATH_UTILITIES_PROTOCOL.IsDevicePathMultiInstance()

Summary

Returns whether a device path is multi-instance.

Prototype

```
typedef
BOOLEAN
(EFI_API *EFI_DEVICE_PATH_UTILS_IS_MULTI_INSTANCE) (
    IN CONST EFI_DEVICE_PATH_PROTOCOL      *DevicePath
);
```

Parameters

DevicePath

Points to the device path. If NULL, then ignored.

Description

This function returns whether the specified device path has multiple path instances.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL* is defined in *EFI Device Path Protocol*.

Returns

This function returns **TRUE** if the device path has more than one instance or **FALSE** if it is empty or contains only a single instance.

10.6 EFI Device Path Display Format Overview

This section describes the recommended conversion between an EFI Device Path Protocol and text. It also describes standard protocols for implementing these. The goals are:

- Standardized display format. This allows documentation and test tools to understand output coming from drivers provided by multiple vendors.
- Increase Readability. Device paths need to be read by people, so the format should be in a form which can be deciphered, maintaining as much as possible the industry standard means of presenting data. In this case, there are two forms, a display-only form and a parse-able form.
- Round-trip conversion from text to binary form and back to text without loss, if desired.

- Ease of command-line parsing. Since device paths can appear on the command-lines of UEFI applications executed from a shell, the conversion format should not prohibit basic command-line processing, either by the application or by a shell.

10.6.1 Design Discussion

The following subsections describe the design considerations for conversion to and from the EFI Device Path Protocol binary format and its corresponding text form.

10.6.1.1 Standardized Display Format

Before the UEFI 2.0, there was no standardized format for the conversion from the EFI Device Path protocol and text. Some de-facto standards arose, either as part of the standard implementation or in descriptive text in the EFI Device Driver Writer's Guide, although they didn't agree. The standardized format attempts to maintain at least the spirit of these earlier ideas.

10.6.1.2 Readability

Since these are conversions to text and, in many cases, users have to read and understand the text form of the EFI Device Path, it makes sense to make them as readable as reasonably possible. Several strategies are used to accomplish this:

- Creating simplified forms for well-known device paths. For example, a PCI root Bridge can be represented as `Acpi(PNP0A03,0)`, but makes more sense as `PciRoot(0)`. When converting from text to binary form, either form is accepted, but when converting from binary form to text, the latter is preferred.
- Omitting the conversion of fields which have empty or default values. By doing this, the average display length is greatly shortened, which improves readability.

10.6.1.3 Round-Trip Conversion

The conversions specified here guarantee at least that conversion to and from the binary representation of the EFI Device Path will be semantically identical.

Text-to-Binary-Conversion

$\text{Text}_1 \rightarrow \text{Binary}_1 \rightarrow \text{Text}_2 \rightarrow \text{Binary}_2$

In the above conversion, the process described in this section is applied to `Text1`, converting it to `Binary1`. Subsequently, `Binary1` is converted to `Text2`. Finally, the `Text2` is converted to `Binary2`. In these cases, `Binary1` and `Binary2` will always be identical. `Text1` and `Text2` may or may not be identical. This is the result of the fact that the text representation has, in some cases, more than one way of representing the same EFI Device Path node.

Binary to Text Conversion

$\text{Binary}_1 \rightarrow \text{Text}_1 \rightarrow \text{Binary}_2 \rightarrow \text{Text}_2$

In the above conversion, the process described in this section is applied to `Binary1`, converting it to `Text1`. Subsequently, `Text1` is converted to `Binary2`. Finally, `Binary2` is converted to `Text2`. In these cases, `Binary1` and `Binary2` will always be identical and `Text1` and `Text2` will always be identical.

Another consideration in round-trip conversion is potential ambiguity in parsing. This happens when the text representation could be converted into more than type of device node, thus requiring information beyond that contained in the text representation in order to determine the correct conversion to apply. In the case of EFI Device Paths, this causes problems primarily with literal strings in the device path, such as those found in file names, volumes or directories.

For example, the file name Acpi(PNP0A03,0) might be a legal FAT32 file name. However, in parsing this, it is not clear whether it refers to an Acpi device node or a file name. Thus, it is ambiguous. In order to prevent ambiguity, certain characters may only be used for device node keywords and may not be used in file names or directories.

10.6.1.4 Command-Line Parsing

Applications written to this specification need to accept the text representation of EFI device paths as command-line parameters, possibly in the context of a command-prompt or shell. In order to do this, the text representation must follow simple guidelines concerning its format.

Command-line parsing generally involves three separate concepts: substitution, redirection and division.

In substitution, the invoker of the application modifies the actual contents of the command-line before it is passed to the application. For example:

```
copy *.xyz
```

In redirection, the invoker of the application gleans from the command line parameters which it uses to, for example, redirect or pipe input or output. For example:

```
echo This text is copied to a file >abc
dir | more
```

Finally, in division, the invoker or the application startup code divides the command-line up into individual arguments. The following line, for example, has (at least) three arguments, divided by whitespace:

```
copy /b file1.info file2.info
```

10.6.1.5 Text Representation Basics

This section describes the basic rules for the text representation of device nodes and device paths. The formal grammar describing appears later.

The text representation of a device path (or text device path) consists of one or more text device nodes, each preceded by a '/' or '\' character. The behavior of a device path where the first node is not preceded by one of these characters is undefined. Some implementations may treat it as a relative path from a current working directory.

Spaces are not allowed at any point within the device path, except when contained in double quotes ("). The '|' (bar), '<' (less than) and '>' (greater than) characters are likewise reserved for use by the shell.

Device Path Text Representation

```
device-path:=\device-node
/device-node
\device-path device-node
/device-path device-node
```

There are two types of text device nodes : file-name/directory or canonical. Canonical text device nodes are prefixed by an option name consisting of only alphanumerical characters, followed by a parenthesis, followed by option-specific parameters separated by a ',' (comma). File names and directories have no prefixes.

Text Device Node Names

```
device-node :=standard-device-node | file-name/directory
standard-device-node :=option-name(option-parameters)
file-name/directory := any character except '/'\"'\"'|'>'<'
```

The canonical device node can have zero or more option parameters between the parentheses. Multiple option parameters are separated by a comma. The meaning of the option parameters depends primarily on the option name, then the parameter-identifier (if present) and then the order of appearance in the parameter list. The parameter identifier allows the text representation to only contain the non-default option parameter value, even if it would normally appear fourth in the list of option parameters. Missing parameters do not require the comma unless needed as a placeholder to correctly increment the parameter count for a subsequent parameter.

Consider:

```
AcpiEx(HWP0002, PNP0A03,0)
```

Which could also be written:

```
AcpiEx(HWP0002,CID=PNP0A03) or
```

```
AcpiEx(HWP0002,PNP0A03)
```

Since CID and UID are optional parameters. Or consider:

```
Acpi(HWP0002,0)
```

Which could also be written:

```
Acpi(HWP0002)
```

Since UID is an optional parameter.

Device Node Option Names

```
option-name :=alphanumeric characters string
option-parameters :=option-parameter
option-parameters,option-parameter
option-parameter :=parameter-value
parameter-identifier=parameter-value
```

10.6.1.6 Text Device Node Reference

In each of the following table rows, a specific device node type and sub-type are given, along with the most general form of the text representation. Any parameters for the device node are listed in italics. In each case, the type is listed and along with it what is required or optional, and any default value, if applicable. On subsequent lines, alternate representations are listed. In general, these alternate representations are simplified by the assumption that one or more of the parameters is set to a specific value.

Parameter Types

This section describes the various types of option parameter values.

Table 10.66: EFI Device Path Option Parameter Values

GUID	An EFI GUID in standard format xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx. (See Appendix A)
Keyword	In some cases, one of a series of keywords must be listed.
Integer	Unless otherwise specified, this indicates an unsigned integer in the range of 0 to 2 ³² -1. The value is decimal, unless preceded by “0x” or “0X”, in which case it is hexadecimal.
EISAID	A seven character text identifier in the format used by the ACPI specification. The first three characters must be alphabetic, either upper or lower case. The second four characters are hexadecimal digits, either numeric, upper case or lower case. Optionally, it can be the number 0.
String	Series of alphabetic, numeric and punctuation characters not including a right parenthesis ‘)’, bar ‘
HexDump	Series of bytes, represented by two hexadecimal characters per byte. Unless otherwise indicated, the size is only limited by the length of the device node.
IPv4 Address	Series of four integer values (each between 0-255), separated by a ‘.’ Optionally, followed by a ‘:’ and an integer value between 0-65555. If the ‘:’ is not present, then the port value is zero.
IPv6 Address	IPv6 Address is expressed in the format [address]:port. The ‘address’ is expressed in the way defined in RFC4291 Section 2.2. The ‘:port’ after the [address] is optional. If present, the ‘port’ is an integer value between 0-65535 to represent the port number, or else, port number is zero.

Table 10.67: Device Node Table

Device Type/SubType/Other	Node	Description
(when type is not recognized)		<i>Path (type, subtype, data)</i> The type is an integer from 0-255. The sub-type is an integer from 0-255. The data is a hex dump.
Type: 1 (Hardware Device Path) (when subtype is not recognized)		<i>HardwarePath (subtype, data)</i> The subtype is an integer from 0-255. The data is a hex dump.
Type: 1 (Hardware Device Path) SubType: 1 (PCI)		<i>Pci(Device, Function)</i> The Device is an integer from 0-31 and is required. The Function is an integer from 0-7 and is required.

continues on next page

Table 10.67 – continued from previous page

Type: 1 (Hardware Device Path) SubType: 2 (PcPcard)	<i>PcCard</i> (Function) The Function is an integer from 0-255 and is required.
Type: 1 (Hardware Device Path) SubType: 3 (Memory Mapped)	MemoryMapped (<i>EfiMemoryType, StartingAddress, EndingAddress</i>) The <i>EfiMemoryType</i> is a 32-bit integer and is required. The <i>StartingAddress</i> and <i>EndingAddress</i> are both 64-bit integers and are both required.
Type: 1 (Hardware Device Path) SubType: 4 (Vendor)	<i>VenHw</i> (Guid, Data) The Guid is a GUID and is required. The Data is a Hex Dump and is optional. The default value is zero bytes.
Type: 1 (Hardware Device Path) SubType: 5 (Controller)	<i>Ctrl</i> (Controller) Controller is an integer and is required.
Type: 1 (Hardware Device Path) SubType: 6 (BMC)	<i>BMC</i> (Type,Address) The Type is an integer from 0-255, and is required. The Address is an unsigned 64-bit integer, and is required.
Type: 2 (ACPI Device Path) (when subtype is not recognized)	<i>AcpiPath</i> (subtype, data) The subtype is an integer from 0-255. The data is a hex dump.
Type: 2 (ACPI Device Path) SubType: 1 (ACPI Device Path)	<i>Acpi</i> (HID,UID) The HID parameter is an EISAID and is required. The UID parameter is an integer and is optional. The default value is zero.
Type: 2 (ACPI Device Path) SubType: 1 (ACPI Device Path) HID=PNP0A03	<i>PciRoot</i> (UID) The UID parameter is an integer. It is optional but required for display. The default value is zero.

continues on next page

Table 10.67 – continued from previous page

Type: 2 (ACPI Device Path) SubType: 1 (ACPI Device Path) HID=PNP0A08	PcieRoot(UID) The UID parameter is an integer. It is optional but required for display. The default value is zero.
Type: 2 (ACPI Device Path) SubType: 1 (ACPI Device Path) HID=PNP0604	Floppy(UID) The UID parameter is an integer. It is optional for input but required for display. The default value is zero.
Type: 2 (ACPI Device Path) SubType: 1 (ACPI Device Path) HID=PNP0301	Keyboard(UID) The UID parameter is an integer. It is optional for input but required for display. The default value is 0.
Type: 2 (ACPI Device Path) SubType: 1 (ACPI Device Path) HID=PNP0501	Serial(UID) The UID parameter is an integer. It is optional for input but required for display. The default value is 0.
Type: 2 (ACPI Device Path) SubType: 1 (ACPI Device Path) HID=PNP0401	ParallelPort(UID) The UID parameter is an integer. It is optional for input but required for display. The default value is 0.
Type: 2 (ACPI Device Path) SubType: 2 (ACPI Expanded Device Path)	AcpiEx(HID,CID,UID,HIDSTR,CIDSTR,UIDSTR) AcpiEx(HID ,(CID Only) The HID parameter is an EISAID. The default value is 0. Either HID or HIDSTR must be present. The CID parameter is an EISAID. The default value is 0. Either CID must be 0 or CIDSTR must be empty. The UID parameter is an integer. The default value is 0. Either UID must be 0 or UIDSTR must be empty. The HIDSTR is a string. The default value is the empty string. Either HID or HIDSTR must be present. The CIDSTR is a string. The default value is an empty string. Either CID must be 0 or CIDSTR must be empty. The UIDSTR is a string. The default value is an empty string. Either UID must be 0 or UIDSTR must be empty.

continues on next page

Table 10.67 – continued from previous page

Type: 2 (ACPI Device Path) SubType: 2 (ACPI Expanded Device Path) HIDSTR=empty CIDSTR=empty UID STR!=empty	AcpiExp(HID,CID,UIDSTR) The HID parameter is an EISAID. It is required. The CID parameter is an EISAID. It is optional and has a default value of 0. The UIDSTR parameter is a string. If UID is 0 and UIDSTR is empty, then use AcpiEx format.
PciRoot(UID UIDSTR) (Display Only)	
Type: 2 (ACPI Device Path) SubType: 2 (ACPI Expanded Device Path) HID=PNP0A03 or CID=PNP0A03 and HID != PNP0A08.	
PcieRoot(UID UIDSTR) (Display Only)	
Type: 2 (ACPI Device Path) SubType: 2 (ACPI Expanded Device Path) HID=PNP0A08 or CID=PNP0A08.	
Type: 2 (ACPI Device Path) SubType: 3 (ACPI ADR Device Path)	AcpiAdr(DisplayDevice[, DisplayDevice...]) The <i>DisplayDevice</i> parameter is an Integer. There may be one or more, separated by a comma.
Type: 2 (ACPI Device Path) SubType: 4 (ACPI NVDIMM Device Path)	NvdimmAcpiAdr(NFIT Device Handle) The NFIT Device Handle is an integer, the _ADR of the NVDIMM device
Type: 3 MessagingPath (when subtype is not recognized)	Msg(<i>subtype</i> , <i>data</i>) The <i>subtype</i> is an integer from 0-255. The <i>data</i> is a hex dump.

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path) SubType: 1 (ATAPI)	Ata (<i>Controller,Drive,LUN</i>) Ata (LUN) (Display only) The Controller is either an integer with a value of 0 or 1 or else the keyword Primary (0) or Secondary (1). It is required. The Drive is either an integer with the value of 0 or 1 or else the keyword Master (0) or Slave (1). It is required. The LUN is a 16-bit integer. It is required.
Type: 3 (Messaging Device Path) SubType: 2 (SCSI)	Scsi (<i>PUN,LUN</i>) The *PUN* is an integer between 0 and 65535 and is required. The LUN is an integer between 0 and 65535 and is required.
Type: 3 (Messaging Device Path) SubType: 3 (Fibre Channel)	Fibre (WWN,LUN) The WWN is a 64-bit unsigned integer and is required. The LUN is a 64-bit unsigned integer and is required.
Type: 3 (Messaging Device Path) SubType: 21 (Fibre Channel Ex)	FibreEx (WWN,LUN) The WWN is an 8 byte array that is displayed in hexadecimal format with byte 0 first (i.e., on the left) and byte 7 last (i.e., on the right), and is required. The LUN is an 8 byte array that is displayed in hexadecimal format with byte 0 first (i.e., on the left) and byte 7 last (i.e., on the right), and is required.
Type: 3 (Messaging Device Path) SubType: 4 (I394)	I1394 (<i>GUID</i>) The GUID is a GUID and is required.
Type: 3 (Messaging Device Path) SubType: 5 (USB)	USB (<i>Port,Interface</i>) The Port is an integer between 0 and 255 and is required. The Interface is an integer between 0 and 255 and is required.
Type: 3 (Messaging Device Path) SubType: 6 (I2O)	I2O (<i>TID</i>) The TID is an integer and is required.

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path) SubType: 9 (Infiniband)	Infiniband (<i>Flags, Guid, ServiceId, TargetId, DeviceId</i>) <i>Flags</i> is an integer. <i>Guid</i> is a guid. <i>ServiceId</i> , <i>TargetId</i> and <i>DeviceId</i> are 64-bit unsigned integers. All fields are required.
Type: 3 (Messaging Device Path) SubType: 10 (Vendor)	VenMsg(Guid, Data) The Guid is a GUID and is required. The Data is a Hex Dump and is option. The default value is zero bytes.
Type: 3 (Messaging Device Path) SubType: 10 (Vendor) GUID= EFI_PC_ANSI_GUID	VenPcAnsi()
Type: 3 (Messaging Device Path) SubType: 10 (Vendor) GUID= EFI_VT_100_GIUD	VenVt100 ()
Type: 3 (Messaging Device Path) SubType: 10 (Vendor) GUID= EFI_VT_100_PLUS_GUID	VenVt100Plus ()
Type: 3 (Messaging Device Path) SubType: 10 (Vendor) GUID= EFI_VT_UTF8_GUID	VenUtf8 ()

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path) SubType: 10 (Vendor) GUID=DEVICE _PATH_MESSAGING _UART_FLOW _CONTROL	UartFlowCtrl (<i>Value</i>) The <i>Value</i> is either an integer with the value 0, 1 or 2 or the keywords XonXoff (2) or Hardware (1) or None (0).
Type: 3 (Messaging Device Path) SubType: 10 (Serial Attached SCSI) Vendor GUID: d487ddb4-008b-11d9-afdc-001083ffc4d	SAS (<i>Address, LUN, RTP, SASSATA, Location, Connect, DriveBay, Reserved</i>) The Address is a 64-bit unsigned integer representing the SAS Address and is required. The LUN is a 64-bit unsigned integer representing the Logical Unit Number and is optional. The default value is 0. The RTP is a 16-bit unsigned integer representing the Relative Target Port and is optional. The default value is 0. The SASSATA is a keyword SAS or SATA or NoTopology or an unsigned 16-bit integer and is optional. The default is NoTopology. If NoTopology or an integer are specified, then Location, Connect and DriveBay are prohibited. If SAS or SATA is specified, then Location and Connect are required, but DriveBay is optional. If an integer is specified, then the topology information is filled with the integer value. The Location is an integer between 0 and 1 or else the keyword Internal (0) or External (1) and is optional. If SASSATA is an integer or NoTopology, it is prohibited. The default value is 0. The Connect is an integer between 0 and 3 or else the keyword Direct (0) or Expanded (1) and is optional. If SASSATA is an integer or NoTopology, it is prohibited. The default value is 0. The DriveBay is an integer between 1 and 256 and is optional unless SASSATA is an integer or NoTopology, in which case it is prohibited. The Reserved field is an integer and is optional. The default value is 0.
Type: 3 (Messaging Device Path) SubType: 10 (Vendor) GUID=EFI_DEBUGPORT _PROTOCOL_GUID	DebugPort()
Type: 3 (Messaging Device Path) SubType: 11 (MAC Address)	MAC(MacAddr, IfType) The MacAddr is a Hex Dump and is required. If IfType is 0 or 1, then the MacAddr must be exactly six bytes. The IfType is an integer from 0-255 and is optional. The default is zero.

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path) SubType: 12 (IPv4)	IPv4(RemoteIp, Protocol, Type, LocalIp, GatewayIPAddress, SubnetMask) IPv4(RemoteIp) (Display Only) The RemoteIp is an IP Address and is required. The Protocol is an integer between 0 and 255 or else the keyword UDP (17) or TCP (6). The default value is UDP. The Type is a keyword, either Static (1) or DHCP (0). It is optional. The default value is DHCP. The LocalIp is an IP Address and is optional. The default value is all zeroes. The GatewayIPAddress is an IP Address and is optional. The default value is all zeroes. The SubnetMask is an IP Address and is optional. The default value is all zeroes.
Type: 3 (Messaging Device Path) SubType: 13 (IPv6)	IPv6(RemoteIp, Protocol, IPAddressOrigin, LocalIp, GatewayIPAddress, SubnetMask) IPv6(RemoteIp) (Display Only) The RemoteIp is an IPv6 Address and is required. The Protocol is an integer between 0 and 255 or else the keyword UDP (17) or TCP (6). The default value is UDP. The IPAddressOrigin is a keyword, could be Static (0), StatelessAutoConfigure (1), or StatefulAutoConfigure (2). The LocalIp is the IPv6 Address and is optional. The default value is all zeroes. The GatewayIPAddress is an IP Address. The PrefixLength is the prefix length of the Local IPv6 Address. The GatewayIPAddress is the IPv6 Address of the Gateway.
Type: 3 (Messaging Device Path) SubType: 14 (UART)	Uart(Baud, DataBits, Parity, StopBits) The Baud is a 64-bit integer and is optional. The default value is 115200. The DataBits is an integer from 0 to 255 and is optional. The default value is 8. The Parity is either an integer from 0-255 or else a keyword and should be D (0), N (1), E (2), O (3), M (4) or S (5). It is optional. The default value is 0. The StopBits is a either an integer from 0-255 or else a keyword and should be D (0), 1 (1), 1.5 (2), 2 (3). It is optional. The default value is 0. Parity and StopBits can either be two integers or two keywords. Mixing formats is prohibited.

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path) SubType: 15 (USB Class)	UsbClass (VID,PID,Class,SubClass,Protocol) The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The Class is an integer between 0 and 255 and is optional. The default value is 0xFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path) SubType: 15 (USB Class) Class 1	UsbAudio(VID,PID,SubClass,Protocol) The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path) SubType: 15 (USB Class) Class 2	UsbCDCC ontrol(VID,PID,SubClass,Protocol) The VID is an optional integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an optional integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an optional integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an optional integer between 0 and 255 and is optional. The default value is 0xFF.

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path)	UsbHID(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 3	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	UsbImage(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 6	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	UsbPrinter(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 7	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	UsbMassStorage(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 8	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path)	UsbHub(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 9	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	UsbC DCDData(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 10	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	UsbSmartCard(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 11	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path)	Us bVideo(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 14	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	Us bDiagnostic(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 220	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	UsbWi reless(VID,PID,SubClass,Protocol)
SubType: 15 (USB Class) Class 224	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The SubClass is an integer between 0 and 255 and is optional. The default value is 0xFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	UsbDevic eFirmwareUpdate(VID,PID,Protocol)
SubType: 15 (USB Class) Class 254	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF. The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
SubClass: 1	

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path)	UsbIrdaBridge(VID,PID,Protocol)
SubType: 15 (USB Class)	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF.
Class 254	
SubClass: 2	The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF.
	The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	UsbTestAndMeasurement(VID,PID,Protocol)
SubType: 15 (USB Class)	The VID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF.
Class 254	
SubClass: 3	The PID is an integer between 0 and 65535 and is optional. The default value is 0xFFFF.
	The Protocol is an integer between 0 and 255 and is optional. The default value is 0xFF.
Type: 3 (Messaging Device Path)	UsbWwid(VID,PID,InterfaceNumber,"WWID")
SubType: 16 (USB WWID Class)	The VID is an integer between 0 and 65535 and is required.
	The PID is an integer between 0 and 65535 and is required.
	The InterfaceNumber is an integer between 0 and 255 and is required.
	The WWID is a string and is required.
Type: 3 (Messaging Device Path)	Unit(LUN)
SubType: 17 (Logical Unit Class)	The LUN is an integer and is required.
Type: 3 (Messaging Device Path)	Sata(HPN, PMPN, LUN)
SubType: 18 (SATA)	The HPN is an integer between 0 and 65534 and is required.
	The PMPN is an integer between 0 and 65535 and is optional. If not provided, the default is 0xFFFF, which implies no port multiplier.
	The LUN is a 16-bit integer. It is required. Note that LUN is applicable to ATAPI devices only, and most ATAPI devices assume LUN=0

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path) SubType: 19 (iSCSI)	iSCSI(TargetName, PortalGroup, LUN, HeaderDigest, DataDigest, Authentication, Protocol) The TargetName is a string and is required. PortalGroup is an unsigned 16-bit integer and is required. The LUN is an 8 byte array that is displayed in hexadecimal format with byte 0 first (i.e., on the left) and byte 7 last (i.e., on the right), and is required. The HeaderDigest is a keyword None or CRC32C is optional. The default is None. The DataDigest is a keyword None or CRC32C is optional. The default is None. The Authentication is a keyword None or CHAP_BI or CHAP_UNI and optional. The default is None. The Protocol defines the network protocol used by iSCSI and is optional. The default is TCP.
Type: 3 (Messaging Device Path) SubType: 20 (VLAN)	Vlan (VlanId)
Type: 3 (Messaging Device Path) SubType: 22 (Serial Attached SCSI Ex)	<i>SasEx (Address, LUN, RTP, SASSATA, Location, Connect, DriveBay)</i> The <i>Address</i> is an 8 byte array that is displayed in hexadecimal format with byte 0 first (i.e., on the left) and byte 7 last (i.e., on the right), and is required. The <i>LUN</i> is an 8 byte array that is displayed in hexadecimal format with byte 0 first (i.e., on the left) and byte 7 last (i.e., on the right), and is optional. The default value is 0. The <i>RTP</i> is a 16-bit unsigned integer representing the Relative Target Port and is optional. The default value is 0. The <i>SASSATA</i> is a keyword <i>SA S</i> or <i>SATA</i> or <i>NoTopology</i> or an unsigned 16-bit integer and is optional. The default is <i>NoTopology</i>. If <i>NoTopology</i> or an integer are specified, then <i>Location</i> , <i>Connect</i> and <i>DriveBay</i> are prohibited. If <i>SAS</i> or <i>SATA</i> is specified, then <i>Location</i> and <i>Connect</i> are required, but <i>DriveBay</i> is optional. If an integer is specified, then the topology information is filled with the integer value. The <i>Location</i> is an integer between 0 and 1 or else the keyword <i>Internal</i> (0) or <i>External</i> (1) and is optional. If <i>SASSATA</i> is an integer or <i>NoTopology</i> , it is prohibited. The default value is 0. The <i>Connect</i> is an integer between 0 and 3 or else the keyword <i>Direct</i> (0) or <i>Expanded</i> (1) and is optional. If <i>SASSATA</i> is an integer or <i>NoTopology</i> , it is prohibited. The default value is 0. The <i>DriveBay</i> is an integer between 1 and 256 and is optional unless <i>SASSATA</i> is an integer or <i>NoTopology</i> , in which case it is prohibited.

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path) SubType: 23 (NVM Express Namespace)	NVMe(NSID,EUI) The NSID is a namespace identifier that is displayed in hexadecimal format with an integer value between 0 and 0xFFFFFFFF. The EUI is the IEEE Extended Unique Identifier (EUI-64) that is displayed in hexadecimal format represented as a set of octets separated by dashes (hexadecimal notation), e.g., FF-FF-FF-FF-FF-FF-FF-FF.
Type: 3 (Messaging Device Path) SubType: 24 (URI)	Uri(<i>Uri</i>) The <i>Uri</i> is optional.
Type: 3 (Messaging Device Path) SubType: 25 (Universal Flash Storage)	UFS (<i>PUN,LUN</i>) The <i>PUN</i> is 0 for current UFS2.0 spec. For future UFS specs which support multiple devices on a UFS port, it would reflect the device ID on the UFS port. The <i>LUN</i> is 0-7 for common LUNs or 81h, D0h, B0h and C4h for well-known LUNs supported by UFS.
Type: 3 (Messaging Device Path) SubType: 26 (SD)	SD (<i>Slot Number</i>) <i>SlotNumber</i> is an integer. It is optional and has a default value of 0.
Type: 3 (Messaging Device Path) SubType: 27 (Bluetooth)	Bluetooth(BD_ADDR) <i>BD_ADDR</i> is HEX dump of 48-bit Bluetooth device address.
Type: 3 (Messaging Device Path) SubType: 28 (Wi-Fi)	Wi-Fi (<i>SSID</i>) The <i>SSID</i> is a string and is required.
Type: 3 (Messaging Device Path) SubType: 29 (eMMC)	eMMC (SlotNumber) SlotNumber is an integer. It is optional and has a default value of 0.

continues on next page

Table 10.67 – continued from previous page

Type: 3 (Messaging Device Path) SubType: 30 (BluetoothLE)	BluetoothLE(BD_ADDR, AddressType) BD_ADDR is HEX dump of 48-bit Bluetooth device address. AddressType is an integer.
Type: 3 (Messaging Device Path) SubType: 31 (DNS)	Dns(DnsServerIp[, DnsServerIp...]) DnsServerIp is optional. It is the IP address of DNS server.
Type: 3 (Messaging Device Path) SubType: 32 (NVDIMM Service)	NVDIMM(UUID) Namespace Unique Identifier UUID
Type: 3 (Messaging Device Path) SubType: 33 (REST Service)	RestService(RestExServiceType, AccessMode) For vendor-specific REST service: RestService(RestExServiceType, AccessMode, VendorRestServiceGuid, VendorDefinedData) RestExServiceType is 0xff in this case.
Type: 3 (Messaging Device Path) SubType: 34 (NVMe-oF Namespace)	NVMeoF(SubsystemNQN, NID) SubsystemNQN – Maximum 224 byte unique subsystem identifier including null termination in compliance with the NVM Express® Base Specification. NID – Globally unique identifier defined by the value of the Namespace Identifier (NID) defined by the Namespace Identification Descriptor from the NVM Express® Base Specification.
Type: 4 (Media Device Path) (when subtype is not recognized)	MediaPath(subtype, data) The subtype is an integer from 0-255 and is required. The data is a hex dump.

continues on next page

Table 10.67 – continued from previous page

Type: 4 (Media Device Path) SubType: 1 (Hard Drive)	H D(Partition,Type,Signature,Start, Size) HD(Partition,Type,Signature) (Display Only) The Partition is an integer representing the partition number. It is optional and the default is 0. If Partition is 0, then Start and Size are prohibited. The Type is an integer between 0-255 or else the keyword MBR (1) or GPT (2). The type is optional and the default is 2. Signature is an integer if Type is 1 or else GUID if Type is 2. The signature is required. The Start is a 64-bit unsigned integer. It is prohibited if Partition is 0. Otherwise it is required. The Size is a 64-bit unsigned integer. It is prohibited if Partition is 0. Otherwise it is required.
Type: 4 (Media Device Path) SubType: 2 (CD-ROM)	CDROM(Entry,Start,Size) CDROM(Entry) (Display Only) The Entry is an integer representing the Boot Entry from the Boot Catalog. It is optional and the default is 0. The Start is a 64-bit integer and is required. The Size is a 64-bit integer and is required.
Type: 4 (Media Device Path) SubType: 3 (Vendor)	VenMedia(GUID, Data) The Guid is a GUID and is required. The Data is a Hex Dump and is option. The default value is zero bytes.
Type: 4 (Media Device Path) SubType: 4 (File Path)	String The String is the file path and is a string.
Type: 4 (Media Device Path) SubType: 5 (Media Protocol)	Media(Guid) The Guid is a GUID and is required.

continues on next page

Table 10.67 – continued from previous page

Contents are defined in the UEFI <i>PI Specification</i> .	
Type: 4 (Media Device Path) SubType: 6 (PIWG Firmware File)	
Contents are defined in the UEFI <i>PI Specification</i> .	
Type: 4 (Media Device Path) SubType: 7 (PIWG Firmware Volume)	
Type: 4 (Media Device Path) SubType: 8 (Relative Offset Range)	<i>Offset (StartingOffset,EndingOffset)</i> The <i>StartingOffset</i> is an unsigned 64-bit integer. The <i>EndingOffset</i> is an unsigned 64-bit integer.
Type: 4 (Media Device Path) SubType: 9 (RAM Disk)	<i>RamDisk (StartingAddress , EndingAddress , DiskInstance , DiskTypeGuid)</i> The <i>StartingAddress</i> and <i>EndingAddress</i> are both 64-bit integers and are both required. The <i>DiskInstance</i> is a 16-bit integer and is optional. The default value is 0. The <i>DiskTypeGuid</i> is a GUID and is required.
Type: 4 (Media Device Path) SubType: 9 (RAM Disk) Disk Type GUID= EFI_VIRTUAL_DISK_GUID	<i>VirtualDisk StartingAddress , EndingAddress , DiskInstance)</i> The <i>StartingAddress</i> and <i>EndingAddress</i> are both 64-bit integers and are both required. The <i>DiskInstance</i> is a 16-bit integer and is optional. The default value is 0.
Type: 4 (Media Device Path) SubType: 9 (RAM Disk) Disk Type GUID= EFI_VIRTUAL_CD_GUID	<i>VirtualCD (StartingAddress , EndingAddress , DiskInstance)</i> The <i>StartingAddress</i> and <i>EndingAddress</i> are both 64-bit integers and are both required. The <i>DiskInstance</i> is a 16-bit integer and is optional. The default value is 0.

continues on next page

Table 10.67 – continued from previous page

Type: 4 (Media Device Path) SubType: 9 (RAM Disk) Disk Type GUID= EFI_PERSISTENT _VIRTUAL_DISK _GUID	<i>PersistentVirtualDisk</i> (<i>StartingAddress</i> , <i>EndingAddress</i> , <i>DiskInstance</i>) The <i>StartingAddress</i> and <i>EndingAddress</i> are both 64-bit integers and are both required. The <i>DiskInstance</i> is a 16-bit integer and is optional. The default value is 0.
Type: 4 (Media Device Path) SubType: 9 (RAM Disk) Disk Type GUID= EFI_PERSISTENT _VIRTUAL_CD_GUID	<i>PersistentVirtualCD</i> (<i>StartingAddress</i> , <i>EndingAddress</i> , <i>DiskInstance</i>) The <i>StartingAddress</i> and <i>EndingAddress</i> are both 64-bit integers and are both required. The <i>DiskInstance</i> is a 16-bit integer and is optional. The default value is 0.
Type: 5 (Media Device Path) (when subtype is not recognized)	<i>BbsPath</i> (subtype, data) The subtype is an integer from 0-255. The data is a hex dump.
Type: 5 (BIOS Boot Specification Device Path) SubType: 1 (BBS 1.01)	<i>BBS</i> (Type,Id,Flags) <i>BBS</i> (Type, Id) (Display Only) The Type is an integer from 0-65535 or else one of the following keywords: Floppy (1), HD (2), CDROM (3), PCMCIA (4), USB (5), Network (6). It is required. The Id is a string and is required. The Flags are an integer and are optional. The default value is 0.

10.6.2 Device Path to Text Protocol

10.6.2.1 EFI_DEVICE_PATH_TO_TEXT_PROTOCOL

Summary

Convert device nodes and paths to text.

GUID

```
#define EFI_DEVICE_PATH_TO_TEXT_PROTOCOL_GUID \
{0x8b843e20, 0x8132, 0x4852, \
 {0x90, 0xcc, 0x55, 0x1a, 0x4e, 0x4a, 0x7f, 0x1c}}
```

Protocol Interface Structure

```
typedef struct _EFI_DEVICE_PATH_TO_TEXT_PROTOCOL {
    EFI_DEVICE_PATH_TO_TEXT_NODE      ConvertDeviceNodeToText;
    EFI_DEVICE_PATH_TO_TEXT_PATH      ConvertDevicePathToText;
} EFI_DEVICE_PATH_TO_TEXT_PROTOCOL;
```

Parameters

ConvertDeviceNodeToText

Converts a device node to text.

ConvertDevicePathToText

Converts a device path to text.

Description

The `EFI_DEVICE_PATH_TO_TEXT_PROTOCOL` provides common utility functions for converting device nodes and device paths to a text representation.

10.6.3 `EFI_DEVICE_PATH_TO_TEXT_PROTOCOL.ConvertDeviceNodeToText()`

Summary

Convert a device node to its text representation.

Prototype

```
typedef
CHAR16*
(EFI_API *EFI_DEVICE_PATH_TO_TEXT_NODE) (
    IN CONST EFI_DEVICE_PATH_PROTOCOL*    DeviceNode,
    IN BOOLEAN                            DisplayOnly,
    IN BOOLEAN                            AllowShortcuts
);
```

Parameters

DeviceNode

Points to the device node to be converted.

DisplayOnly

If *DisplayOnly* is **TRUE**, then the shorter text representation of the display node is used, where applicable. If *DisplayOnly* is **FALSE**, then the longer text representation of the display node is used.

AllowShortcuts

If *AllowShortcuts* is **TRUE**, then the shortcut forms of text representation for a device node can be used, where applicable.

Description

The `ConvertDeviceNodeToText` function converts a device node to its text representation and copies it into a newly allocated buffer.

The *DisplayOnly* parameter controls whether the longer (parseable) or shorter (display-only) form of the conversion is used.

The *AllowShortcuts* is **FALSE**, then the shortcut forms of text representation for a device node cannot be used. A shortcut form is one which uses information other than the type or subtype.

The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in *EFI Device Path Protocol*

Returns

This function returns the pointer to the allocated text representation of the device node data or else NULL if *DeviceNode* was NULL or there was insufficient memory.

10.6.4 EFI_DEVICE_PATH_TO_TEXT_PROTOCOL.ConvertDevicePathToText()

Summary

Convert a device path to its text representation.

Prototype

```
typedef
CHAR16*
(EFIAPI *EFI_DEVICE_PATH_TO_TEXT_PATH) (
    IN CONST EFI_DEVICE_PATH_PROTOCOL    *DevicePath,
    IN BOOLEAN                           DisplayOnly,
    IN BOOLEAN                           AllowShortcuts
);
```

Parameters

DeviceNode

Points to the device path to be converted.

DisplayOnly

If *DisplayOnly* is **TRUE**, then the shorter text representation of the display node is used, where applicable. If *DisplayOnly* is **FALSE**, then the longer text representation of the display node is used.

AllowShortcuts

The *AllowShortcuts* is **FALSE**, then the shortcut forms of text representation for a device node cannot be used.

Description

This function converts a device path into its text representation and copies it into an allocated buffer.

The *DisplayOnly* parameter controls whether the longer (parseable) or shorter (display-only) form of the conversion is used.

The *AllowShortcuts* is **FALSE**, then the shortcut forms of text representation for a device node cannot be used. A shortcut form is one which uses information other than the type or subtype.

The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL* is defined in *EFI Device Path Protocol*.

Returns

This function returns a pointer to the allocated text representation of the device node or NULL if *DevicePath* was NULL or there was insufficient memory.

10.6.5 Device Path from Text Protocol

10.6.5.1 EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL

Summary

Convert text to device paths and device nodes.

GUID

```
#define EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL_GUID \
    {0x5c99a21,0xc70f,0x4ad2,\
     {0x8a,0x5f,0x35,0xdf,0x33,0x43,0xf5, 0x1e}}
```

Protocol Interface Structure

```
typedef struct _EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL {
    EFI_DEVICE_PATH_FROM_TEXT_NODE      ConvertTextToDeviceNode;
    EFI_DEVICE_PATH_FROM_TEXT_PATH      ConvertTextToDevicePath;
} EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL;
```

Parameters

ConvertTextToDeviceNode

Converts text to a device node.

ConvertTextToDevicePath

Converts text to a device path.

Description

The *EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL* provides common utilities for converting text to device paths and device nodes.

10.6.5.2 EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL.ConvertTextToDeviceNode()

Summary

Convert text to the binary representation of a device node.

Prototype

```
typedef
EFI_DEVICE_PATH_PROTOCOL*
(EFI_API *EFI_DEVICE_PATH_FROM_TEXT_NODE) (
    IN CONST CHAR16*      TextDeviceNode
);
```

Parameters

TextDeviceNode

TextDeviceNode points to the text representation of a device node. Conversion starts with the first character and continues until the first non-device node character.

Description

This function converts text to its binary device node representation and copies it into an allocated buffer.

The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in *EFI Device Path Protocol*.

Returns

This function returns a pointer to the EFI device node or NULL if *TextDeviceNode* is NULL or there was insufficient memory.

10.6.5.3 EFI_DEVICE_PATH_FROM_TEXT_PROTOCOL.ConvertTextToDevicePath()

Summary

Convert a text to its binary device path representation.

Prototype

```
typedef
EFI_DEVICE_PATH_PROTOCOL*
(EFI_API *EFI_DEVICE_PATH_FROM_TEXT_PATH) (
    IN CONST CHAR16*           TextDevicePath
);
```

Parameters

TextDevicePath

TextDevicePath points to the text representation of a device path. Conversion starts with the first character and continues until the first non-device path character.

Description

This function converts text to its binary device path representation and copies it into an allocated buffer.

The memory is allocated from EFI boot services memory. It is the responsibility of the caller to free the memory allocated.

Related Definitions

EFI_DEVICE_PATH_PROTOCOL is defined in *EFI Device Path Protocol*.

Returns

This function returns a pointer to the allocated device path or NULL if *TextDevicePath* is NULL or there was insufficient memory.